

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-add16b832e68cb70f.jsonl`
- SHA-256: `13a5a40a9d497b98ec52b8f00a132e7ac4592697a1a3f6619bec75f68a171dc6`
- Source modified: `2026-06-21T13:56:46+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T13:52:09.111Z)

Research solving the DUAL-PYTHON-ENV problem (main torch 2.6 + WASB native runner needs torch 1.11 in a separate env). Options: (a) conda-pack / constructor, (b) micromamba self-contained env, (c) run WASB as a subprocess against its own venv (how native_wasb_runner already shells out), (d) port WASB to torch 2.x (effort/risk?), (e) Docker-only for the native model, (f) make the LIGHT tier Gemini-only (no torch at all) and the HEAVY tier a separate download. License check on conda (Anaconda ToS!) vs conda-forge/miniforge/micromamba. Recommend the cleanest path for a downloadable app. Use web search.

Return the RESEARCH schema. Context about our app:

PROJECT: a local, AI-powered badminton/racket-sports HIGHLIGHT INDEXER + rally detector. Engine repo (PRIMARY): C:/Users/avidu/Projects/badminton-highlight-indexer (Python, FastAPI + SQLite + ffmpeg + GPU CV/AI; paid Gemini = cloud AI). Sibling repos: C:/Users/avidu/Projects/khelsutra (the web/ React SPA + site/ marketing), C:/Users/avidu/Projects/rally-annotator (VLC plugin, MIT), C:/Users/avidu/Projects/sports-data-collector, C:/Users/avidu/Projects/sports-obsreport.

GOAL of the plan we are building: bake the engine into a DOWNLOADABLE, "batteries-included" Python APP that a user can download and run WITHOUT sourcing the repo (no git clone / build-from-source). It must spin up a local WEB SERVICE that runs (a) INFERENCE (video -> rallies -> highlight reel), (b) TRAINING (BYO model on their own labelled data), and (c) RALLY ANNOTATION if needed -- runnable BOTH locally and against CLOUD services. Preserve every already-completed end-to-end CUJ.

KEY FACTS the orchestrator already verified:

```
- pyproject.toml: hatchling build, namespace package (no backend/__init__.py),
packages=["backend","training"], console script: indexer = backend.main:main. requires-python
>=3.10.
- torch/torchvision are DELIBERATELY NOT in dependencies -- their CUDA/CPU wheels need --index-url (PEP621 cannot express). Optional extras: gpu(empty, docs-only), auth(PyJWT), reporting(git+ssh obsreport), storage-s3(boto3), storage-gcs(google-cloud-storage). google-genai is a hard dep.
- Server today: python -m backend.main --server --port 8000 . Worker: python -m backend.worker {infer|train} . CLI single-shot: python -m backend.main --video-path ... --segmenter {gemini|yolo_hybrid|wasb_hybrid|motion}.
- THE HARD PACKAGING PROBLEM: native WASB shuttle model runs in a SEPARATE Python env (conda, torch 1.11+cu113) while the main engine uses torch 2.6+cu124. ffmpeg/ffprobe are system deps. Model weights (wasb_badminton_best.pth.tar) live in a GCS bucket.
- Decoupled-compute SEAM already built (M1-M8 merged, default-OFF): deployment.profile + compute_target on AppConfig; backend/config/presets.py (PROFILE_PRESETS: truly-local / cloud-serving / cpu-mock); backend/storage/* (local/gcs/s3/gdrive StorageBackend registry); backend/compute/* (ComputeBackend registry, CloudRunCompute dispatches a Cloud Run Job); backend/config/startup.py per-profile checks; backend/billing.py cost ledger.
- The web SPA lives in a DIFFERENT repo (khelsutra/web, Vite/React/TS/Tailwind, i18n en+kn+hi).
```

The engine has a likely-DEAD backend/api/static. Bundling the UI is a cross-repo concern.

- Licensing guardrail (NON-NEGOTIABLE): every dependency -- code, data, AND weights -- must be MIT/Apache-2.0/BSD only. Paid LLMs = inference/serving only, never training data. Any NEW packaging dependency must also be license-clean.
- A prior docs/DESKTOP_APP_PLAN.md exists but was SUPERSEDED by owner decision D10: deliver a "self-hosted webserver + UI the user spins up" (NOT a desktop app).

khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md is the live appliance design.

Always ground claims in file:line or a doc path. Read the actual files; do not assume.

2. assistant (2026-06-21T13:52:12.820Z)

I'll research the dual-Python-env problem. Let me start by reading the relevant files in the codebase to ground my analysis, then do web research on the packaging options.

3. user (2026-06-21T13:52:15.074Z)

```
tests\test_calibrate_wasb.py
backend\detectors\__pycache__\wasb_runner.cpython-313.pyc
tests\__pycache__\test_calibrate_wasb.cpython-313-pytest-9.0.3.pyc
output\wasb_tuned_segments.csv
output\wasb_tuned_segments_baseline.csv
output\wasb_tuned_segments_vs_golden.csv
backend\detectors\__pycache__\wasb_infer.cpython-313.pyc
tests\test_export_wasb_segments.py
tests\__pycache__\test_export_wasb_segments.cpython-313-pytest-9.0.3.pyc
backend\eval\export_wasb_segments.py
tests\__pycache__\test_calibrate_wasb.cpython-313.pyc
tests\__pycache__\test_export_wasb_segments.cpython-313.pyc
tests\__pycache__\test_wasb_cache.cpython-313.pyc
backend\eval\calibrate_wasb.py
output\wasb\mahadevpura_smoke_180s__05e0e577029d_wasb.csv
output\wasb\GX010125_1080p__89605e80ed01_wasb.csv
.claude\worktrees\bhi-execpolicy\backend\eval\calibrate_wasb.py
.claude\worktrees\bhi-execpolicy\backend\eval\export_wasb_segments.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\detectors\wasb_runner.py
.claude\worktrees\bhi-execpolicy\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\bhi-execpolicy\tests\test_calibrate_wasb.py
.claude\worktrees\bhi-execpolicy\tests\test_config_wasb.py
.claude\worktrees\bhi-execpolicy\tests\test_export_wasb_segments.py
.claude\worktrees\bhi-execpolicy\tests\test_wasb_cache.py
.claude\worktrees\bhi-execpolicy\tests\test_wasb_runner.py
.claude\worktrees\bhi-
execpolicy\tests\__pycache__\test_calibrate_wasb.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\bhi-execpolicy\tests\__pycache__\test_config_wasb.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\bhi-
execpolicy\tests\__pycache__\test_export_wasb_segments.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\bhi-execpolicy\tests\__pycache__\test_wasb_cache.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\bhi-execpolicy\tests\__pycache__\test_wasb_runner.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\bhi-execpolicy\backend\eval\__pycache__\calibrate_wasb.cpython-313.pyc
.claude\worktrees\bhi-execpolicy\backend\eval\__pycache__\export_wasb_segments.cpython-313.pyc
.claude\worktrees\bhi-
execpolicy\backend\pipeline\detectors\__pycache__\wasb_infer.cpython-313.pyc
.claude\worktrees\bhi-
execpolicy\backend\pipeline\detectors\__pycache__\wasb_runner.cpython-313.pyc
.claude\worktrees\bhi-
execpolicy\backend\pipeline\segmenters\__pycache__\wasb_hybrid.cpython-313.pyc
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\calibrate_wasb.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval\export_wasb_segments.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_calibrate_wasb.py
.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_export_wasb_segments.py
.claude\worktrees\agent-a771ff1d85e18f2ed\tests\__pycache__\test_calibrate_wasb.cpython-313-
pytest-9.0.3.pyc
.claude\worktrees\agent-a771ff1d85e18f2ed\tests\__pycache__\test_export_wasb_segments.cpython-
```

313-pytest-9.0.3.pyc
.git\logs\refs\heads\fix\wasb-test-mocks-readability-preflight
.git\refs\heads\fix\wasb-test-mocks-readability-preflight
tests\test_wasb_runner.py
tests__pycache__\test_wasb_runner.cpython-313-pytest-9.0.3.pyc
output\wasb_sxs\png\mahadevpura_smoke_180s__05e0e577029d_wasb.csv
output\wasb_sxs\stream\mahadevpura_smoke_180s__05e0e577029d_wasb.csv
.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_wasb_runner.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_runner.py
.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_config_wasb.py
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\agent-a771ff1d85e18f2ed\tests\test_wasb_cache.py
.git\logs\refs\heads\perf\wasb-fast-frames
.git\refs\heads\perf\wasb-fast-frames
.claude\worktrees\agent-a771ff1d85e18f2ed\tests__pycache__\test_config_wasb.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\agent-a771ff1d85e18f2ed\tests__pycache__\test_wasb_cache.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\agent-a771ff1d85e18f2ed\tests__pycache__\test_wasb_runner.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval__pycache__\calibrate_wasb.cpython-313.pyc
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\eval__pycache__\export_wasb_segments.cpython-313.pyc
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors__pycache__\wasb_infer.cpython-313.pyc
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\detectors__pycache__\wasb_runner.cpython-313.pyc
.claude\worktrees\agent-a771ff1d85e18f2ed\backend\pipeline\segmenters__pycache__\wasb_hybrid.cpython-313.pyc
backend\pipeline\detectors\wasb_runner.py
.claude\worktrees\tier1-windowing\backend\eval\calibrate_wasb.py
.claude\worktrees\tier1-windowing\backend\eval\export_wasb_segments.py
.claude\worktrees\tier1-windowing\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\tier1-windowing\backend\pipeline\detectors\wasb_runner.py
.claude\worktrees\tier1-windowing\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\tier1-windowing\tests\test_calibrate_wasb.py
.claude\worktrees\tier1-windowing\tests\test_config_wasb.py
.claude\worktrees\tier1-windowing\tests\test_export_wasb_segments.py
.claude\worktrees\tier1-windowing\tests\test_wasb_cache.py
.claude\worktrees\tier1-windowing\tests\test_wasb_runner.py
.claude\worktrees\tier1-windowing\tests__pycache__\test_calibrate_wasb.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\tier1-windowing\tests__pycache__\test_config_wasb.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\tier1-windowing\tests__pycache__\test_export_wasb_segments.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\tier1-windowing\tests__pycache__\test_wasb_cache.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\tier1-windowing\tests__pycache__\test_wasb_runner.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\tier1-windowing\backend\eval__pycache__\calibrate_wasb.cpython-313.pyc
.claude\worktrees\tier1-windowing\backend\eval__pycache__\export_wasb_segments.cpython-313.pyc
.claude\worktrees\tier1-windowing\backend\pipeline\detectors__pycache__\wasb_infer.cpython-313.pyc
.claude\worktrees\tier1-windowing\backend\pipeline\detectors__pycache__\wasb_runner.cpython-313.pyc
.claude\worktrees\tier1-windowing\backend\pipeline\segmenters__pycache__\wasb_hybrid.cpython-313.pyc
.claude\worktrees\w2-safeguard\backend\eval\calibrate_wasb.py
.claude\worktrees\w2-safeguard\backend\eval\export_wasb_segments.py
.claude\worktrees\w2-safeguard\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\w2-safeguard\backend\pipeline\detectors\wasb_runner.py
.claude\worktrees\w2-safeguard\backend\pipeline\segmenters\wasb_hybrid.py
.claude\worktrees\w2-safeguard\tests\test_calibrate_wasb.py
.claude\worktrees\w2-safeguard\tests\test_config_wasb.py

```
.claude\worktrees\w2-safeguard\tests\test_export_wasb_segments.py
.claude\worktrees\w2-safeguard\tests\test_wasb_cache.py
.claude\worktrees\w2-safeguard\tests\test_wasb_runner.py
.claude\worktrees\w2-safeguard\tests\__pycache__\test_calibrate_wasb.cpython-313-pytest-
9.0.3.pyc
.claude\worktrees\w2-safeguard\tests\__pycache__\test_config_wasb.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\w2-safeguard\tests\__pycache__\test_export_wasb_segments.cpython-313-pytest-
9.0.3.pyc
.claude\worktrees\w2-safeguard\tests\__pycache__\test_wasb_cache.cpython-313-pytest-9.0.3.pyc
.claude\worktrees\w2-safeguard\tests\__pycache__\test_wasb_runner.cpython-313-pytest-9.0.3.pyc
(Results are truncated. Consider using a more specific path or pattern.)
```

4. user (2026-06-21T13:52:15.323Z)

```
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/calibrate_wasb.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/export_wasb_segments.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/eval/__pycache__/calibrate_wasb.cpython-
313.pyc
./.claude/worktrees/agent-
a771ff1d85e18f2ed/backend/eval/__pycache__/export_wasb_segments.cpython-313.pyc
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/wasb_infer.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/detectors/wasb_runner.py
./.claude/worktrees/agent-
a771ff1d85e18f2ed/backend/pipeline/detectors/__pycache__/wasb_infer.cpython-313.pyc
./.claude/worktrees/agent-
a771ff1d85e18f2ed/backend/pipeline/detectors/__pycache__/wasb_runner.cpython-313.pyc
./.claude/worktrees/agent-a771ff1d85e18f2ed/backend/pipeline/segmenters/wasb_hybrid.py
./.claude/worktrees/agent-
a771ff1d85e18f2ed/backend/pipeline/segmenters/__pycache__/wasb_hybrid.cpython-313.pyc
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_calibrate_wasb.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_config_wasb.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_export_wasb_segments.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_wasb_cache.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/test_wasb_runner.py
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/__pycache__/test_calibrate_wasb.cpython-313-
pytest-9.0.3.pyc
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/__pycache__/test_config_wasb.cpython-313-
pytest-9.0.3.pyc
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/__pycache__/test_export_wasb_segments.cpython-
313-pytest-9.0.3.pyc
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/__pycache__/test_wasb_cache.cpython-313-
pytest-9.0.3.pyc
./.claude/worktrees/agent-a771ff1d85e18f2ed/tests/__pycache__/test_wasb_runner.cpython-313-
pytest-9.0.3.pyc
./.claude/worktrees/bhi-execpolicy/backend/eval/calibrate_wasb.py
./.claude/worktrees/bhi-execpolicy/backend/eval/export_wasb_segments.py
./.claude/worktrees/bhi-execpolicy/backend/eval/__pycache__/calibrate_wasb.cpython-313.pyc
./.claude/worktrees/bhi-execpolicy/backend/eval/__pycache__/export_wasb_segments.cpython-313.pyc
./.claude/worktrees/bhi-execpolicy/backend/pipeline/detectors/wasb_infer.py
./.claude/worktrees/bhi-execpolicy/backend/pipeline/detectors/wasb_runner.py
./.claude/worktrees/bhi-
execpolicy/backend/pipeline/detectors/__pycache__/wasb_infer.cpython-313.pyc
./.claude/worktrees/bhi-
execpolicy/backend/pipeline/detectors/__pycache__/wasb_runner.cpython-313.pyc
./.claude/worktrees/bhi-execpolicy/backend/pipeline/segmenters/wasb_hybrid.py
./.claude/worktrees/bhi-
execpolicy/backend/pipeline/segmenters/__pycache__/wasb_hybrid.cpython-313.pyc
./.claude/worktrees/bhi-execpolicy/tests/test_calibrate_wasb.py
./.claude/worktrees/bhi-execpolicy/tests/test_config_wasb.py
./.claude/worktrees/bhi-execpolicy/tests/test_export_wasb_segments.py
./.claude/worktrees/bhi-execpolicy/tests/test_wasb_cache.py
./.claude/worktrees/bhi-execpolicy/tests/test_wasb_runner.py
./.claude/worktrees/bhi-
execpolicy/tests/__pycache__/test_calibrate_wasb.cpython-313-pytest-9.0.3.pyc
./.claude/worktrees/bhi-
```

```

execpolicy/tests/__pycache__/test_config_wasb.cpython-313-pytest-9.0.3.pyc
./.claude/worktrees/bhi-
execpolicy/tests/__pycache__/test_export_wasb_segments.cpython-313-pytest-9.0.3.pyc
./.claude/worktrees/bhi-
execpolicy/tests/__pycache__/test_wasb_cache.cpython-313-pytest-9.0.3.pyc
./.claude/worktrees/bhi-
execpolicy/tests/__pycache__/test_wasb_runner.cpython-313-pytest-9.0.3.pyc
./.claude/worktrees/boxhill17/backend/eval/calibrate_wasb.py
./.claude/worktrees/boxhill17/backend/eval/export_wasb_segments.py
./.claude/worktrees/boxhill17/backend/pipeline/detectors/wasb_infer.py
./.claude/worktrees/boxhill17/backend/pipeline/detectors/wasb_runner.py
./.claude/worktrees/boxhill17/backend/pipeline/detectors/__pycache__/wasb_runner.cpython-313.pyc
./.claude/worktrees/boxhill17/backend/pipeline/segmenters/wasb_hybrid.py
./.claude/worktrees/boxhill17/backend/pipeline/segmenters/__pycache__/wasb_hybrid.cpython-
313.pyc
./.claude/worktrees/boxhill17/output/wasb
./.claude/worktrees/boxhill17/output/wasb/Boxhill_Doubles_proxy__9a35ffdfbde_wasb.csv
./.claude/worktrees/boxhill17/output/wasb/Boxhill_Doubles_proxy__9a35ffdfbde_wasb_wasbcache

```

5. assistant (2026-06-21T13:52:19.562Z)

Let me read the actual WASB runner files in the primary repo (not the worktrees) to understand the current subprocess approach.

6. user (2026-06-21T13:52:20.521Z)

```

1      """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3      Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4          1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5          2. copies wasb_infer.py into the WASB repo's src/ dir,
6          3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7          4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9      So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10     with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11     """
12     import os
13     import shlex
14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21     TrackNetRunner.
22     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
24     wsl_tilde_quote
25
26     logger = logging.getLogger(__name__)
27
28     # Windows path to the inference wrapper we stage into WSL.
29     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
30
31     @dataclass
32     class WasbConfig:
33         """Resolved settings for a WASB WSL run (built from config.json ->
34         indexing_wasb)."""
35         distro: str = "Ubuntu"
36         repo_dir: str = "~/models/WASB-SBDT"
37         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
38         conda_env: str = "wasb"

```

```

37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43         # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44         # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45         # extraction that dominates a long clip. Output is bit-identical to the PNG path
46         # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47         stream_video: bool = False
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                   f"python -c \"import torch; print('torch', torch.__version__, "
76                   f"\"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."
83             out = (res.stdout or "").strip()
84             if res.returncode != 0:
85                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86             return True, out
87
88         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89             src_mnt = to_wsl_mnt_path(video_win_path)
90             reason = self.wsl_source_unreadable_reason(src_mnt)
91             if reason:
92                 logger.error("Cannot stage video into WSL: %s", reason)
93                 return None
94             dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"

```

```

96         try:
97             res = self._wsl_bash(cmd)
98         except subprocess.TimeoutExpired:
99             logger.error("Staging video into WSL timed out.")
100             return None
101         if res.returncode != 0 or "OK" not in (res.stdout or ""):
102             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103             return None
104         return dest
105
106     def _stage_infer_script(self) -> bool:
107         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108         src_mnt = to_wsl_mnt_path(_INFER_WIN)
109         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110         res = self._wsl_bash(cmd)
111         return res.returncode == 0 and "OK" in (res.stdout or "")
112
113     def run_predict(self, video_win_path: str, output_win_dir: str,
114                     video_id: Optional[str] = None) -> Optional[str]:
115         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
116
117         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
118         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
119         can never reuse each other's staged frames, resumable cache, or CSV.
120
121         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
122         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
123         # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
124         # the CSV landed inside WSL while Windows polled the repo-relative path.
125         output_win_dir = os.path.abspath(output_win_dir)
126         os.makedirs(output_win_dir, exist_ok=True)
127         # Normalize for cross-platform basename (tests use Windows paths on Linux).
128         video_win_path = str(video_win_path).replace("\\", "/")
129         base = os.path.basename(video_win_path)
130         stem, ext = os.path.splitext(base)
131         # Unique, content-derived key: same filename + different bytes -> different key.
132         key = f"{stem}__{video_id[:12]}" if video_id else stem
133         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136         if not self._stage_infer_script():
137             logger.error("Could not stage wasb_infer.py into WSL.")
138             return None
139         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140         if staged_video is None:
141             return None
142         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145         # WSL stage filesystem is already low so the user can clean up before it fills.
146         # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
147         if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
148             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
149             if free is not None and free < self.cfg.min_free_gb:
150                 logger.warning(
151                     "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
152                     "may fill the disk ? consider running tools/cleanup_caches.py.",
153                     free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min

```

```

clip.
157         if self.cfg.stream_video:
158             frame_args = "--stream-video "
159         else:
160             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161         cmd = (
162             f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163             f"python wasb_infer.py "
164             f"--video {wsl_tilde_quote(staged_video)} "
165             f"{frame_args}"
166             f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167             f"--sport {shlex.quote(self.cfg.sport)} "
168             f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv'}}"
169         )
170         logger.info("Running WASB inference on %s ...", base)
171         try:
172             res = self._wsl_bash(cmd)
173         except subprocess.TimeoutExpired:
174             logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175             return None
176         if res.returncode != 0:
177             logger.error("WASB inference failed:\n%s", (res.stderr or
178 res.stdout)[-2000:])
179             return None
180         if not os.path.exists(expected_csv_win):
181             logger.error("WASB finished but expected CSV not found at %s",
182 expected_csv_win)
183             return None
184         logger.info("WASB trajectory CSV: %s", expected_csv_win)
185
186         # Success ? the trajectory CSV is the durable output; the staged video copy and
187         # the (huge) decoded frame cache are now pure intermediates. Delete them unless
188         # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
189         # so the frame cache + detector cache remain for a resumable re-run.
190         # (Streaming never writes a frame cache, so there's only the staged video to
191         drop.)
192         if not self.cfg.keep_frames:
193             stale = [staged_video] if self.cfg.stream_video else [frames_dir,
194 staged_video]
195             logger.info("Cache hygiene: removing staged video%s for %s "
196                 "(set indexing.wasb.keep_frames=true to retain).",
197                 "" if self.cfg.stream_video else " + frame cache", key)
198             self._wsl_rm_rf(stale)
199             return expected_csv_win
200
201         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
202         agnostic).
203         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
204         trajectory_to_action_windows =
205         staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

7. user (2026-06-21T13:52:20.945Z)

File does not exist. Note: your current working directory is C:\Users\avidu\Projects\badminton-highlight-indexer.

8. assistant (2026-06-21T13:52:24.725Z)

Now I understand the current architecture: WASB runs via WSL2 subprocess in a conda `wasb` env (torch 1.11+cu113), shelling out from the Windows-side main engine (torch 2.6+cu124). Let me read the relevant config, the wasb_infer.py, the appliance architecture doc, and check the pyproject.

9. user (2026-06-21T13:52:25.695Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36     python wasb_infer.py --frames_dir /path/frames --weights
37     ../pretrained_weights/wasb_badminton_best.pth.tar \
38     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39     every 1000] [--fresh]
40     """
41     import os
42     import os.path as osp
43     import csv
44     import glob
45     import json
46     import argparse
47     import logging
48     from collections import defaultdict
49     from contextlib import contextmanager
50
51     logger = logging.getLogger(__name__)
52
53     CACHE_VERSION = 1
54     _DET_FILE = "det_raw.jsonl"
55     _MANIFEST_FILE = "manifest.json"
56
57     def _build_cfg(weights: str, sport: str, device: str = "cuda"):
58         """Compose the WASB eval config via Hydra, overriding model/weights/device.
59         ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity
60         preserved), ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests
```

```
``runner.gpus=[0]``;
```

10. user (2026-06-21T13:52:26.375Z)

```
1      # PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
2      #
3      # Build backend: hatchling. `backend/` is an intentional NAMESPACE package
4      # (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
5      # reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
6      # auto-detection would otherwise miss the un-__init__.d `backend` root.
7      #
8      # torch/torchvision are deliberately NOT listed in [project.dependencies]:
9      # their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
10     # which PEP 621 cannot express. Install them separately ? see [project.optional-
dependencies]
11     # `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
12     [build-system]
13     requires = ["hatchling"]
14     build-backend = "hatchling.build"
15
16     [project]
17     name = "badminton-highlight-indexer"
18     version = "0.1.0"
19     description = "AI-assisted rally segmentation and highlight indexing for racket sports."
20     readme = "README.md"
21     requires-python = ">=3.10"
22     license = { text = "MIT" }
23     authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
24     keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]
25
26     # Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
27     dependencies = [
28         "fastapi>=0.111.0",
29         "uvicorn>=0.30.1",
30         "opencv-python>=4.9.0.80",
31         "numpy>=1.26.4",
32         "pydantic>=2.7.1",
33         "jinja2>=3.1.4",
34         "python-dotenv>=1.0.0",
35         "google-genai>=2.4.0",
36         "supervision>=0.21.0,<0.30.0",
37     ]
38
39     [project.optional-dependencies]
40     # Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
41     dev = [
42         "pytest",
43         "pytest-cov",
44         "httpx",          # fastapi TestClient transport
45         "ruff",
46         "mypy",
47     ]
48
49     # Per-video observability reports (SOFT dependency: the pipeline degrades to a
50     # no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
51     # fetched over SSH; if you lack access, simply omit this group ? reporting
52     # tests skip via pytest.importorskip. For local dev against a sibling checkout:
53     # pip install -e ../sports-obsreport --no-build-isolation
54     reporting = [
55         "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
56     ]
57
58     # `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
59     # CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
60     # express, so they are NOT declarable here. Install them out-of-band instead:
```

```

61     # # CUDA 12.4 box:
62     # pip install torch>=2.12.0 torchvision>=0.27.0 --index-url
https://download.pytorch.org/whl/cu124
63     # # CPU-only box (also what CI uses):
64     # pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
65     gpu = []
66
67     # Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
68     # these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
69     # install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare
R2 +
70     # DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
71     # MOUNTED Google Drive (Google Drive for Desktop) as plain files; see
backend/storage/gdrive.py.
72     storage-s3 = ["boto3>=1.34"]
73     storage-gcs = ["google-cloud-storage>=2.16"]
74
75     [project.scripts]
76     # `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
77     indexer = "backend.main:main"
78
79     [tool.hatch.metadata]
80     # The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
81     # reference*); hatchling requires opting in to allow those in metadata.
82     allow-direct-references = true
83
84     [tool.hatch.build.targets.wheel]
85     # `backend` is a namespace package (no __init__.py); list both import roots
86     # explicitly so an editable/wheel install exposes `import backend` and
87     # `import training` regardless of cwd.
88     packages = ["backend", "training"]
89

```

11. assistant (2026-06-21T13:52:29.561Z)

Now let me read the appliance architecture doc and the WASB config/segmenter to understand how WASB is wired, and check what torch versions the WASB native model truly needs.

12. user (2026-06-21T13:52:32.192Z)

```

ANNOTATE_AND_TRAIN.md
DRY_RUN_CUJ.md
FRIENDLY_AND_MULTILINGUAL.md
LOCAL_APPLIANCE_ARCHITECTURE.md
PER_RALLY_SELECTION.md
PORTABILITY.md
README.md
REQUEST_ACCESS_FORM.md
---DESKTOP---
DESKTOP_APP_PLAN.md
VIDEO_LOCALITY_MODEL.md

```

13. user (2026-06-21T13:52:32.534Z)

```

1     """WASB-based hybrid play segmenter.
2
3     Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
4     WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
5     badminton weights and runs today, so this is the live shuttle-motion substrate.
6
7     For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
8     candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
9     min_window_duration) are what get tuned against golden rally labels.
10

```

```

11     The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
12     lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
13     WASB runner and identity.
14     """
15     from typing import Any, Dict, Tuple
16
17     from backend.pipeline.segmenters.base import segmenter_registry
18     from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
19     from backend.pipeline.detectors.base import DetectorRunner
20     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
21
22
23     class WasbHybridSegmenter(TrajectoryHybridSegmenter):
24         family = "wasb"
25         display_label = "WASB"
26
27         @property
28         def name(self) -> str:
29             return "wasb_hybrid"
30
31         @property
32         def description(self) -> str:
33             return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained
badminton "
34                     "weights) for precise candidate rallies + AI provider for semantic
boundaries.")
35
36         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
Any]]:
37             cfg = WasbConfig.from_indexing_cfg(idx_cfg)
38             return WasbRunner(cfg), {"weights_path": cfg.weights_path}
39
40         def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner,
Dict[str, Any]]:
41             # Lazy import: the native runner pulls no heavy deps, but keep parity with the
stub
42             # seam (imported only when detector_impl='native' is actually selected).
43             from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner,
NativeWasbConfig
44             cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
45             return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
46                                           "detector_impl": "native", "device": cfg.device}
47
48
49     segmenter_registry.register(WasbHybridSegmenter)
50

```

14. assistant (2026-06-21T13:52:35.207Z)

There's a `native\_wasb\_runner.py`. The orchestrator's question mentions option (c) "how native_wasb_runner already shells out". Let me read that file and the appliance doc.

15. user (2026-06-21T13:52:36.155Z)

```

1     """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3     The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4     "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
M3.5).
5     It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
6     Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
core
7     that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8     see the acceptance gate below) ? but WITHOUT the WSL transport:

```

```

9
10     * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11     ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12     (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13     * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
14     ``auto``
15     (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
16     ``cuda`` stays
17     strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
18     ``runner.device`` override).
19     * weights / repo / python / scratch come from the **environment** first
20     (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
21     ``indexing.wasb.`` config, then a default ? so a box needs no config edits.
22     It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
23     name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
24     ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
25     the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
26     (``trajectory_hybrid.resolve_runner`` ? ``_build_native_runner``);
27     ``detector_impl="native"``.
28
29     **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
30     runner
31     (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
32     same
33     candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
34     deterministic
35     ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
36     exact
37     run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
38     streaming
39     vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
40     artifact), and two native runs were byte-identical (deterministic). This is a hardware
41     check
42     (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
43     contract,
44     not the numbers.
45     """
46     import os
47     import shutil
48     import logging
49     import subprocess
50     from dataclasses import dataclass
51     from typing import Any, Dict, List, Optional, Tuple
52
53     from backend.pipeline.detectors.base import DetectorRunner
54     from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
55     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
56     WSL/TrackNet.
57     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
58
59     logger = logging.getLogger(__name__)
60
61     # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
62     # detectors/trackers/data loaders + find configs/), exactly as the WSL adapter does.
63     _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
64
65     # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
66     probe
67     # so the "cuda True/False" string they parse can never drift apart.
68     _CUDA_PROBE_CODE = "import torch; print('torch', torch.__version__, 'cuda',
69     torch.cuda.is_available())"
70
71     @dataclass

```

```

60     class NativeWasbConfig:
61         """Resolved settings for a Linux-native WASB run.
62
63         Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
64         over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
65         ``device`` defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
66         perf win).
67         """
68         repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
69         python_bin: str = "python" # the `wasb` conda env python (invoked by
70         path, no activate)
71         weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or
72         indexing.wasb.weights_path)
73         sport: str = "badminton"
74         device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-
75         available-else-cpu)
76         scratch_dir: str = "" # frame-extraction scratch (env); "" = next
77         to the output dir
78         timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no
79         timeout)
80         keep_frames: bool = False # retain the decoded frame cache after
81         success (debug only)
82         # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
83         clip's
84         # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
85         real GPU
86         # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
87         here.
88         stream_video: bool = True
89
90     @classmethod
91     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
92         w = (idx_cfg or {}).get("wasb", {}) or {}
93         env = os.environ.get
94         # stream_video is tri-state in config: None/absent => native default (stream);
95         an
96         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
97         sv = w.get("stream_video")
98         return cls(
99             repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
100             python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
101             weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
102             sport=w.get("sport", "badminton"),
103             device=env("WASB_DEVICE") or w.get("device", "cuda"),
104             scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
105             "",
106             timeout_sec=int(w.get("timeout_sec", 1800)),
107             keep_frames=bool(w.get("keep_frames", False)),
108             stream_video=(True if sv is None else bool(sv)),
109         )
110
111     class NativeWasbRunner(DetectorRunner):
112         """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
113         docstring."""
114
115         def __init__(self, cfg: NativeWasbConfig):
116             self.cfg = cfg
117             # Cached CUDA-availability probe (used to resolve device="auto"). Set by
118             healthcheck;
119             # lazily probed by run_predict if healthcheck was skipped. None = not yet
120             probed.
121             self._cuda_available: Optional[bool] = None

```

```

109         # --- low-level native exec (the single place tests patch)
-----
110         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
111             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
112             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
113                                   timeout=(self.cfg.timeout_sec or None))
114
115         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
116         def _probe_cuda(self) -> bool:
117             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
118             (missing python / timeout / import error) is treated as 'no CUDA'."""
119             try:
120                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
121             except (FileNotFoundError, subprocess.TimeoutExpired):
122                 return False
123             return res.returncode == 0 and "cuda True" in (res.stdout or "")
124
125         def _cuda_is_available(self) -> bool:
126             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
127             if self._cuda_available is None:
128                 self._cuda_available = self._probe_cuda()
129             return self._cuda_available
130
131         def _effective_device(self) -> str:
132             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
133             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
134             if self.cfg.device == AUTO:
135                 return DeviceContext(AUTO, self._cuda_is_available()).effective
136             return self.cfg.device
137
138         def _repo_src(self) -> str:
139             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
140
141         def _copy_infer_script(self) -> bool:
142             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
143             repo_src = self._repo_src()
144             try:
145                 os.makedirs(repo_src, exist_ok=True)
146                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
147                 return True
148             except OSError as e:
149                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
150                 return False
151
152         def _rm_rf(self, path: Optional[str]) -> None:
153             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
154             if path:
155                 shutil.rmtree(path, ignore_errors=True)
156
157         def healthcheck(self) -> Tuple[bool, str]:
158             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
159             if not self.cfg.weights_path:
160                 return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or
indexing.wasb.weights_path).")
161             weights = os.path.expanduser(self.cfg.weights_path)
162             if not os.path.exists(weights):

```

```

163         return False, f"WASB weights not found at {weights}"
164     repo_src = self._repo_src()
165     if not os.path.isdir(repo_src):
166         return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
167                     f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
168     # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
169     # argparse rc=2 from wasb_infer.py at run time.
170     if self.cfg.device not in VALID_DEVICES:
171         return False, (f"unknown device {self.cfg.device!r} ? valid: {'',
'.join(VALID_DEVICES)).")
172     try:
173         res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
174     except FileNotFoundError:
175         return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON)."
176     except subprocess.TimeoutExpired:
177         return False, "native torch healthcheck timed out."
178     out = (res.stdout or "").strip()
179     if res.returncode != 0:
180         return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip())"
181     # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
182     self._cuda_available = "cuda True" in out
183     ctx = DeviceContext(self.cfg.device, self._cuda_available)
184     if ctx.cuda_required_but_missing:
185         # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
186         # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
187         return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
188     if ctx.fell_back:
189         logger.warning("device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO
CPU; WASB "
190                     "inference will be slow. Set indexing.wasb.device=cuda to
require a GPU.", out)
191     return True, out
192
193     def run_predict(self, video_win_path: str, output_win_dir: str,
194                   video_id: Optional[str] = None) -> Optional[str]:
195         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
196
197         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
198         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
199         same downstream contract, so the parity comparison is apples-to-apples.
200         """
201         output_dir = os.path.abspath(output_win_dir)
202         os.makedirs(output_dir, exist_ok=True)
203         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
204         norm = str(video_win_path).replace("\\", "/")
205         stem, _ext = os.path.splitext(os.path.basename(norm))
206         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
207         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
208         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
209
210         if not self._copy_infer_script():
211             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
212             return None
213
214         weights = os.path.expanduser(self.cfg.weights_path)

```



```

215         argv = [self.cfg.python_bin, "wasb_infer.py",
216                 "--video", os.path.abspath(str(video_win_path))]
217         frames_dir: Optional[str] = None
218         if self.cfg.stream_video:
219             # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
220             argv.append("--stream-video")
221         else:
222             scratch = self.cfg.scratch_dir or output_dir
223             frames_dir = os.path.join(scratch, f"{key}_frames")
224             argv += ["--frames_out_dir", frames_dir]
225             # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
226             # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
227             device = self._effective_device()
228             argv += ["--weights", weights,
229                     "--sport", self.cfg.sport,
230                     "--device", device,
231                     "--out", out_csv]
232
233             logger.info("Running native WASB inference (device=%s%s) on %s ...",
234                         device, " [auto?fallback]" if self.cfg.device == AUTO and device !=
"cuda" else "",
235                         os.path.basename(norm))
236             try:
237                 res = self._run(argv, cwd=self._repo_src())
238             except FileNotFoundError:
239                 logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
240                 return None
241             except subprocess.TimeoutExpired:
242                 logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
243                 return None
244             if res.returncode != 0:
245                 logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
246                 return None
247             if not os.path.exists(out_csv):
248                 logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
249                 return None
250             logger.info("WASB trajectory CSV: %s", out_csv)
251
252             # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
253             # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
254             # (Streaming never materializes frames, so there is nothing to drop on that
path.)
255             if not self.cfg.keep_frames and frames_dir:
256                 logger.info("Cache hygiene: removing frame cache for %s "
257                             "(set indexing.wasb.keep_frames=true to retain).", key)
258                 self._rm_rf(frames_dir)
259             return out_csv
260
261         # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
262         parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
263         trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
264

```

```

1      # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3      > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
bridge) . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-20
4      > **Lifecycle:** `DRAFT ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** $9 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
6      > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
screens. Engine-side changes here are **proposals to coordinate** with the active
decoupled/worker sessions ? not written unilaterally.
7      > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
legal/financial advice.
8
9      ---
10
11     ## 0. TL;DR
12     **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
**one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the
degenerate collapse of the same storage/worker split.
13
14     The **single most important caveat:** the engine has **zero app-layer auth and CORS
`***` `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays
bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`,
and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the
tenancy boundary.**
15
16     Two honest hardware truths: (1) the current droplet is **CPU-only** ? real detection
there is **Gemini (cloud)** or **stub (CPU mock)**; **real local-GPU detection needs a GPU
host** (native WASB, in-flight). (2) The owner priority is to **stand up a GPU host ASAP** ?
provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs **in parallel**
with the UI MVP, which ships on Gemini today.
17
18     ## 1. Problem & goal
19     The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker
{infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the
FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified
backend/main.py]`. What's missing is a **console** that lets a non-CLI operator run the whole
loop on a single box, **honestly, behind a gate**.
20
21     **Headline CUJ (owner's words):** a **hosted website** where I pick an input ? **a
Google Drive file, a bucket file (`s3://`/`gs://`), or a local upload** ? it lands in the
**bucket (DO Spaces / GCS)** or locally ? I **generate highlights** on a **GPU-enabled host**
running the real e2e ? I **pick rallies, watch, and download** ? **completely from the UI.**
22
23     **Outcome ("good")**: an operator signs in, picks a source, watches it index, curates
rallies, downloads a reel ? on the **live CPU droplet today** (Gemini/stub), and on a **GPU host
later** (native WASB) with **no UI fork**.
24
25     **Read to get current:** `PER_RALLY_SELECTION.md` ($3/$4/$10 contract anchors), engine
`HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built),
`VIDEO_LOCALITY_MODEL.md` (privacy/retention).
26
27     ## 2. Decisions locked
28     | # | Decision | Source / date | Implication |
29     |---|---|---|---|
30     | D1 | Reverse-proxied **single origin**; engine stays `127.0.0.1:8000` | `[verified
main.py:638]`; 2026-06-20 | CORS `` is moot from the browser ? the proxy is the boundary |
31     | D2 | **Edge auth**, zero engine auth code | `HOSTING_PLAN.md`; 2026-06-20 | Caddy

```

```

basicauth OR cloudflared + Cloudflare Access |
32 | D3 | MVP drives the in-process FastAPI worker loop, `storage=local` | `[verified
job_worker.py:37, local.py:33]`; 2026-06-20 | bucket worker = reserved scale path, out of MVP |
33 | D4 | Embed RallyPicker from `PER_RALLY_SELECTION.md` verbatim | PR [#11];
2026-06-20 | the console is a superset; do not redesign the picker |
34 | D5 | Console lives in the khelsutra web/ React scaffold | `web/README.md`;
2026-06-20 | Cloudflare-Pages-style root, not engine static |
35 | D6 | Compute placement is a setting, rendered from `GET /api/capabilities` |
`[design]`; 2026-06-20 | UI offers ONLY what the box can honestly do |
36 | D7 | Ingest is a source menu (local upload · bucket URI · Drive) normalizing to
one video object | `[verified backend/storage/ref.py parse_uri]`; 2026-06-20 | Drive is a
stub today ? scoped honestly ($4.2) |
37 | D8 | GPU host is provider-agnostic (DO droplet / GCE VM / Cloud Run+GPU); storage
stays the bucket | Owner 2026-06-20 | compute and storage are separate planes; no provider lock-
in |
38 | D9 | Observability is first-class ? structured logs on every new path + a DoD item
| Owner 2026-06-20 | see $5; verified during CUJ replay |
39 | D10 | Delivery = a self-hosted webserver + UI the user spins up (NOT a desktop
app); the user points at local storage + GPU/CPU OR cloud locations and the UI drives the
tool | Owner 2026-06-20 | supersedes `DESKTOP_APP_PLAN.md`; one artifact (this appliance)
runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
40
41 ## 3. Foundation ? what the engine gives us today `[verified this session unless
noted]`
42 - FastAPI loop: POST /api/process (202 + `job_id`) ? poll GET /api/jobs/{id} ?
POST /api/query ? POST /api/compile ? GET /api/highlights/{video_id}/{filename}
(FileResponse) [main.py]. Chunked upload exists ? POST /api/upload/init · PUT
/api/upload/{id}/part/{i} · POST /api/upload/{id}/complete · DELETE /api/upload/{id}
[verified uploads.py:64,89,123,147] ? but the bundled app.js never calls them (it only
hits /api/process + /api/compile [verified app.js:88,373]). So browser upload is client-
wiring only, no engine change.
43 - In-process worker: one module-global ThreadPoolExecutor(max_workers=1)
[verified job_worker.py:37] ? strictly one job at a time; the DB jobs table is the
clock; fail_stale_jobs sweeps queued/running on boot. Gemini chunk concurrency is a
separate knob indexing.ai_max_workers (default 5 [verified models.py:183]; set to 1
to serialize) ? it is not already 1.
44 - Storage: storage.backend default = "local" [verified models.py:298];
LocalStorage.fetch_to_local is identity [verified local.py:33], put no-ops on
src==dst ? so on a single box the worker's pull?pipeline?push collapses to read-local/write-
local. URI scheme parse_uri [verified ref.py]: bare local:// ? local; s3://
(AWS/R2/DO Spaces/MinIO, by endpoint) ; gcs:// gs:// ; gdrive:// ? follow-up stub
(emulated only). Credentials never in config ? boto3 default chain / ~/aws [verified
s3.py].
45 - Compute seam: indexing.detector_impl ? {auto=WSL, native=Linux-native,
stub=CPU mock}; default "auto" [verified models.py:129], resolved in
trajectory_hybrid._resolve_runner [verified :76-85]. Native WASB is in-flight on branch
feat/native-wasb-runner: native_wasb_runner.py exists [verified];
fusion_hybrid._build_native_runner returns NativeWasbRunner [verified
fusion_hybrid.py:86] while trajectory_hybrid._build_native_runner still raises [verified
:57-63]. Acceptance gate = owner byte-parity (WSL CSV == native CSV) on a real GPU box ?
not exercisable on the droplet or in the offline suite. Treat native as not production-
verified.
46 - Worker CLI: python -m backend.worker {infer|train} pulls a --video-uri ?
pipeline ? pushes outputs/<video_id>/{intervals.csv,report.json} [verified
worker/__main__.py:88,149]. The validated GPU-free/key-free combo: detector_impl=stub +
--set indexing.skip_ai_handoff=true + storage.s3.
47 - Gating: cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access
email-OTP; backend trusts Cf-Access-Authenticated-User-Email [HOSTING_PLAN.md]. GET
/api/health now EXISTS (engine M1, [#268]) ? {status, sport, default_segmenter}
[verified main.py:127, corrected 2026-06-21 ? was "does not exist"].
48 - Droplet reality: 168.144.126.176, CPU-only, NO GPU, region blr1; co-hosts
the marketing site (/srv/khelsutra-site, node :8787) and the engine (~/rally); ran the
full train+infer e2e against DO Spaces with the GPU mocked [SETUP_NEW_MACHINE.md,
DECOUPLED_COMPUTE/HANDOFF.md]. Shared testbed: MinIO 9000/9001 + GCS emulator 9023 also run
there.

```

```

49 - **Absent (net-new):** `GET /api/capabilities`, `GET /api/videos` (only single-row
`get_video(id)`), `GET /api/reels/{video_id}`, any source/proxy streaming endpoint, any
**Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.
50
51 ## 4. Design / architecture
52
53 ### 4.1 Topology & request path
54 ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy /
static host** serving the `web/` SPA bundle and proxying `/api/*` ? engine **FastAPI
`127.0.0.1:8000`** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a
GPU host). The firewall blocks inbound `8000` + the test ports `9000/9001/9023`. The marketing
site stays a **separate** systemd unit on a **separate** subdomain.
55
56 ...
57 PUBLIC INTERNET
58 ? (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app
59 ? cloudflared Tunnel + Cloudflare Access (no inbound ports) OR Caddy :443 +
basicauth
60 ?
61 ?????????????????????????????????????????????????????????????????????????????????
62 ? ONE BOX firewall BLOCKS inbound 8000 + 9000/9001/9023 ?
63 ? (2) reverse proxy / static host (Caddy/cloudflared) ?
64 ? /, /assets/* ? web/ React SPA /api/*, /api/highlights/* ? :8000 ?
65 ? ? ONE ORIGIN to the browser ? engine CORS '*' is irrelevant ?
66 ? ? ?
67 ? (3) ENGINE FastAPI (host=127.0.0.1:8000, HARD-bound [main.py:638]) ?
68 ? control: jobs table + ThreadPoolExecutor(max_workers=1) drain ?
69 ? data: uploads.py chunked upload · /api/compile · /api/highlights ?
70 ? ? ?
71 ? (4) STORAGE = LocalStorage (default) fetch=identity · put=no-op(src==dst) ?
72 ? ? ?
73 ? (5) COMPUTE seam: gemini (cloud, key) · stub (CPU mock) [droplet] ?
74 ? native WASB (GPU host) · auto (owner Win+WSL) ?
75 ?????????????????????????????????????????????????????????????????????????????????
76 RESERVED SCALE PATH (out of MVP): console "compute target" selector ? storage=s3 (DO
Spaces)
77 ? SEPARATE GPU host runs `python -m backend.worker infer --video-uri s3:///?` ?
outputs/<vid>/ back
78 (needs a NET-NEW orchestration endpoint reflecting outputs into the jobs/poll
contract)
79 ...
80
81 ### 4.2 Ingest sources (the headline) ? one menu, one normalized video object
82 All sources normalize to **"a video the pipeline can read"** ? a local path (MVP) or a
`StorageRef` (scale):
83
84 | Source | Path today | Status |
85 |---|---|---|
86 | **Local upload** | SPA chunks the file ? `max_part_mb` via the existing
`/api/upload/init\|part\|complete` `[verified uploads.py]` ? server path ? `POST /api/process`.
(Scale: also `StorageBackend.put` to `s3://`/`gcs://`.) | **wire client only** (endpoints exist)
|
87 | **Bucket URI** (`s3://`, `gs://`) | **`POST /api/process` accepts the bucket URI
directly** (resolve ? fetch to scratch ? same pipeline ? API DB) `[verified main.py:159,274,283
? corrected 2026-06-21; was "net-new"]`; the worker CLI also fetches `--video-uri`. So the
**engine endpoint is NOT net-new** ? only the SPA call that POSTs the URI + polls `/api/jobs`
is. *(Default single-user mode; HOSTED mode w/ `allowed_video_root` correctly rejects a bucket
URI 400.)* | **engine: DONE (verified); SPA ingest call: design (= B-P2)** |
88 | **Google Drive** (`gdrive://`) | `parse_uri` recognizes it, but the gdrive backend is
an **emulated stub** `[verified ref.py, DECOUPLED_COMPUTE research]` | **stub ? roadmap;** MVP
requires a Drive?bucket pre-step; do **not** present Drive as live until the backend is real |
89
90 ### 4.3 Compute placement matrix (one UI, surfaced from `/api/capabilities`)
91 | Placement | Runs | Needs | Real/mock | Honest caveat |
92 |---|---|---|---|---|

```

```

93      | **CPU droplet ? gemini** (MVP target) | `segmenter=gemini`, in-proc worker |
`GEMINI_API_KEY`; network; **no GPU** | **REAL** | Cloud brain: uploads ?1280px chunks to Google
? **must be in consent copy**; **no per-video cost ceiling** today |
94      | **CPU droplet ? stub** | trajectory hybrid + `detector_impl=stub`+`skip_ai_handoff` |
nothing | **MOCK** | not real tracking ? UI must label "stub (CPU mock ? not real detection)";
demo/regression only |
95      | **CPU droplet ? motion** | `segmenter=motion` | nothing | REAL, low-quality | only no-
key/no-GPU real segmenter; never surface `server/receiver/events` (fabricated) |
96      | **GPU host ? native WASB** (in-flight) | `wasb`+`detector_impl=native` ?
`NativeWasbRunner` (in-proc torch) | a **GPU host ? DO GPU droplet / GCE GPU VM / Cloud
Run+GPU**; `WASB_WEIGHTS` | **REAL**, on-device | partially wired (fusion builds it, trajectory
refuses); **owner byte-parity gate**; **Cloud Run/GCE need a containerized worker ? no
Dockerfile exists, and WASB's torch-1.11 conda env makes the image non-trivial** |
97      | **Owner Win+WSL ? auto** | `wasb`+`detector_impl=auto` (WSL2+conda) | Windows+WSL2+GPU
| REAL (today) | Windows-only transport; the working real-GPU path for owner verification |
98      | **Remote GPU worker via bucket** (reserved) | console CPU + `storage=s3`; separate GPU
host runs `backend.worker infer` | bucket creds; a GPU host; a **net-new orchestration
endpoint** | REAL (remote) | FastAPI loop and worker CLI are **separate** today; "remote GPU as
a setting" is **design** ? selector shown **disabled** in MVP |
99
100     **Provider note:** **Cloud Run + GPU** (scale-to-zero, pay-per-job) is the strongest fit
for the engine's own **per-video-billing** economics (`DECOUPLED_COMPUTE` #246) ? gated on the
**containerization feasibility check** above.
101
102     ### 4.4 Auth / exposure (non-negotiable invariant)
103     No engine auth, by design; the gate is external. **Invariant (all three must hold):**
engine bound to `127.0.0.1` + firewall closes `8000` + edge gate up. Go-live checklist
`[HOSTING_PLAN.md:102-108]`: Access/basicauth ON; **rotate the chat-shared `GEMINI_API_KEY`**;
set `security.allowed_video_root`; rate-limit; per-user quota.
104
105     ### 4.5 UI operator console (web/ SPA ? superset of RallyPicker)
106     - **S1 Ingest & Process** ? the source menu ($4.2) + a segmenter picker from
`/api/capabilities` ? `POST /api/process`.
107     - **S2 Progress** ? 3s poll of the free-text stage
(`hashing/validating/segmenting/finalizing`), a **"one job at a time"** notice, and the restart-
failure terminal state.
108     - **S3 Rally pick ? reel** ? **embed RallyPicker verbatim** (`PER_RALLY_SELECTION.md`):
honor `min_shot_count`; never render motion `server/receiver/events` or a fake confidence meter.
**Per-rally is ONE screen of the console.**
109     - **S4 Reels / History** ? needs `GET /api/videos` + `GET /api/reels/{video_id}` for
**reload-survival**.
110     - **S5 Settings** ? render `config` + `capabilities`; the compute/storage **target
selector shown DISABLED** ("single-box: local") until the bucket-worker bridge lands.
111
112     ### 4.6 Reuse map
113     **Reused (no re-plumb):** the full `/api` loop; the in-process worker; the chunked
upload endpoints (un-called); LocalStorage default; the detector seam incl. the landing
`NativeWasbRunner`; the **RallyPicker** design + anchors; the `web/` scaffold; the
`HOSTING_PLAN` gate; the `SETUP_NEW_MACHINE` runbook. **Net-new:** Caddyfile + systemd units;
`GET /api/capabilities`; `GET /api/videos`; `GET /api/reels/{video_id}`; `GET /api/health`; the
SPA console screens; ingest-from-bucket wiring; a retention sweep; (reserved) the bucket-worker
bridge.
114
115     ## 5. Observability ? logging & monitoring `[required, first-class]`
116     Launch-readiness directive (owner 2026-06-20): **logs and monitoring taken seriously,
with a full log-analysis pass across every new code path.** The engine already uses a
centralized Python logger `[verified main.py:12-17]` ? carry that discipline into every new
path.
117
118     - **Correlation ID end-to-end.** Thread the existing `job_id` (from `/api/process`)
**plus** an `upload_id`/`request_id` through ingest ? process ? jobs ? compile ? highlights, and
into the worker for the bucket path, so **one CUJ is traceable** across UI ? FastAPI ? worker ?
GPU host. Log it on every line.
119     - **Structured logs.** New endpoints/screens emit structured (JSON-able) logs at **entry
/ result / error** ? not just "it ran". The SPA logs client-side errors + failed fetches with

```

the same correlation id.

```
120 - **Loud failures, never silent drops.** Carry the engine's ethos (#240 "surface failed
chunks loudly") + RallyPicker's `min_shot_count`-warning rule: a partial/empty reel must
**never** look complete; every dropped clip/chunk is logged + surfaced in the UI.
121 - **Health + metrics.** Ship the missing `GET /api/health` `[HOSTING_PLAN.md:42]` and a
thin metrics surface (jobs queued/running/failed, last error, disk free, per-video cost once a
ledger exists) for the engine-mode banner + alerting.
122 - **Aggregation for transient hosts.** A scale-to-zero Cloud Run / GPU host is
**ephemeral** ? logs must ship to a durable sink (the bucket or a log service), not die with the
instance.
123 - **Log-analysis pass before merge (DoD item).** Every PR that adds a code path includes
a check that the path logs meaningfully, and the **logs are inspected during CUJ live-replay**
($8 Layers 173). Ties [[feedback-launch-quality-bar]].
124
125 ## 6. Risk table
126 | Risk | Today | Mitigation |
127 |---|---|---|
128 | No app auth + CORS `` `[main.py:42]` | open if any of {localhost-bind, firewall,
gate} slips | enforce all three; **LIVE posture test** that raw `:8000` is refused from outside
|
129 | Test ports on shared droplet | MinIO `9000/9001` + GCS emu `9023` | firewall closes
them |
130 | Retention | `output/`+`uploads/` originals **never deleted** | sweep / keep-proxy-only
before any non-owner exposure |
131 | Surprise Gemini/GPU bill | **no cost ceiling/ledger** | per-video cap;
`ai_max_workers` (default 5) tunable; `max_workers=1` job serialization |
132 | Native WASB unverified | partially wired; owner-parity-gated | treat as not-
production-verified until owner byte-parity on a real GPU box |
133 | New engine endpoints | `capabilities`/`videos`/`reels`/`health` don't exist |
coordinate engine PRs with active sessions; **branch from `origin/master`, re-verify HEAD,
explicit `git add`** |
134 | Drive ingest overstated | `gdrive://` is a stub | don't present Drive as live; require
Drive?bucket pre-step until real |
135 | No Dockerfile | runbook-only reproducibility | feasibility-check a worker image before
promising Cloud Run/GCE |
136 | Shared-checkout collisions | active worker sessions share the engine tree | the
branch-safety protocol on every engine-side PR |
137 | **Provisioning-credential leak** | a DO API token / cloud creds could leak if pasted
in chat or committed (cf. the **already-rotated Spaces key** exposed in chat,
`DECOUPLED_COMPUTE/HANDOFF.md`) | authenticate the CLI **on-box** (`doctl auth init` interactive
/ `gcloud` already authed) so the secret never enters chat or the repo; least-privilege token,
revoke after; creds 0600, never committed; **creating paid GPU infra is a confirm-first action
every time** |
138
139 ## 7. Honest limits ? what this does NOT do
140 Does **not** add multi-tenant identity/RLS (single-tenant alpha; uploads/outputs
global). Does **not** run real WASB on the CPU droplet (Gemini or stub only). Does **not** make
the remote-GPU bucket path work (design; selector disabled). Does **not** provide in-UI source
playback, job cancel, or a true progress bar (coarse free-text stage, poll-only). Does **not**
containerize (runbook; no Dockerfile `[verified]`). Does **not** verify native WASB (owner byte-
parity on a real GPU box, not this offline suite). A green SPA + contract tests prove the
**console plumbing**, not engine rally-detection quality (a separate track).
141
142 ## 8. CUJs & live-test plan `[required]`
143 **CUJs:** **CUJ-1** ingest (Drive/bucket/local) ? index ? reel ? download (core);
**CUJ-2** re-open a past match after reload; **CUJ-3** pick-by-query (deterministic filters +
honesty-hiding); **CUJ-4** settings/compute placement (truthful capabilities).
144
145 **Live-test plan ? 5 layers, every CUJ replayed LIVE throughout development:**
146 - **L1 ? Mock engine (hermetic, fast, default + CI).** A tiny FastAPI/MSW implementing
the **exact** contract (upload; `process` 202; `jobs` state machine incl. restart-failure;
`query` with a motion fixture for honesty-hiding; `config`; `capabilities` per box profile ?
CPU-no-key / CPU-with-key / GPU; `compile`; `highlights` tiny mp4; the new `videos`/`reels`).
The SPA drives **all 4 CUJs in <1s**, **replayed via the Claude Preview/browser tools** (start
dev server ? fill the ingest picker ? click through process/poll/RallyPicker ? assert the reel
```

link via snapshot/network). Catches reload-survival (CUJ-2), the `play\_segments` field-name bug (CUJ-3), capability honesty (CUJ-4).

147 - **L2 ? Engine-contract checks (real FastAPI, GPU-free).** Boot `python -m backend.main --server` with `storage=local` + `detector\_impl=stub` + `skip\_ai\_handoff` and assert the **mock** and the real engine agree shape-for-shape** (202 keys, jobs enum, `result.play\_segments` schema, compile `{status,filepath}`, highlights content-type). Every new endpoint is born with a mock handler **and** a real-engine contract test **in** the same PR**. Runs in CI.

148 - **L3 ? Real local e2e (the MVP substrate).** Through the gate on the droplet: CUJ-1 with `stub` (key-free) and `gemini` (real, short clip to bound cost); the **exposure-safety posture test** (raw `:8000`/`:900x` refused from outside) as an assertion, not an assumption; reuse the `parity.yml` dual-host pattern.

149 - **L4 ? CI regression (green gates merge).** `npm test` + a stub-engine contract job + headless browser-replay CUJ scripts; one small PR per tracker row off `origin/master`, **no stacking**; coverage at/above the floor; **logs inspected during replay** (S5).

150 - **L5 ? Owner hardware e2e (handed off).** Native WASB **byte-parity** on a real GPU box; real-4K **golden-corpus** runs (Gemini + native); Modern-Standby/cv2-vs-ffmpeg behavior; a multi-GB browser upload. Reported back into S9, stated plainly as **not verified** in the offline suite.

151

152 **## 9. Deliverables & progress tracker ? source of truth**

153 Legend: ? Todo · ? In progress · ? Done · ? Blocked/gated. **One small PR per row**, branched from `origin/master`, no stacking; CI green gates merge; coverage ? floor; each PR ships its tests + meaningful logs (S5).

154

ID	Deliverable	Depends on	Gated?	Status	PR
T0	This design doc + `docs/README.md` + `NEXT_STEPS.md` pointers			?	owner
T1	Mock engine + capability fixtures + browser-replay CUJ scripts (L1)			no	?
T2	Appliance shell: Caddyfile + systemd units; firewall; edge auth ; rotate key (P1)			?	owner
T3	L3 live posture test (raw `:8000`/`:900x` refused from outside)			no	?
T4	S1 ingest (local upload wired) + segmenter picker			no	?
T5	S2 progress (poll + one-job + restart-failure)			no	?
T6	S3 embed RallyPicker + compile + download			no	?
T7	(engine) `GET /api/capabilities` + `GET /api/health` + banner			coord	?
T8	(engine) `GET /api/videos` + `GET /api/reels/{video_id}` (S4 reload-survival)			coord	?
T9	S5 Settings (config+capabilities; disabled compute/storage target)			no	?
T10	Observability pass: correlation id end-to-end + structured logs + metrics (S5)			no	?
T11	Retention sweep for `output/`+`uploads/`			owner	?
T12	Ingest from bucket URI (`s3:///`/`gs:///`) wiring ? engine half DONE (`/api/process` accepts URIs, M2/[#280], `[verified]`); remaining = the SPA ingest call (= B-P2)			coord	?
T13	GPU host stand-up (provider-agnostic) + wire `detector_impl=native`; owner byte-parity (P4)			owner	?
T14	(reserved) Bucket-worker bridge ? remote-GPU "compute as a setting" (P5)			owner	?

172

173 **Owner priority (2026-06-20):** T13 (GPU host) is elevated to run in parallel from the start ? owner provisions the host + token; the engine session lands native WASB; the UI MVP does **not** block on it (ships on Gemini).

174

175 **## 10. Open questions ? owner / external**

176 **Blocking gates:** (1) **edge-auth** ? Caddy basicauth (self-contained) vs cloudflared+Cloudflare Access (matches the two-host design, no inbound ports)? (2) **retention defaults** ? auto-delete originals after compile? keep proxies+intervals only? window? (privacy/consent). (3) **per-video Gemini/GPU cost ceiling** ? what cap, hard-fail vs warn? (4) **GPU host choice** ? DO GPU droplet / GCE GPU VM / Cloud Run+GPU (and is a containerized worker worth building for Cloud Run)? (5) **P5 bridge ownership** ? who builds the orchestration

```

endpoint + how it coordinates with the active worker sessions.
177     **Nice-to-knows:** console home (web/ vs engine static); a review-playback streaming
endpoint; per-user scoping timing; containerization vs runbook.
178
179     ## 11. Definition of done
180     - [ ] CUJ-1 works end to end on the droplet (Gemini **and** stub) behind the gate, via
the ingest menu.
181     - [ ] Raw `:8000`/`:9000` unreachable from outside (**tested**, not assumed).
182     - [ ] Local upload wired; bucket-URI ingest at least designed/tracked; Drive scoped
honestly (not faked).
183     - [ ] RallyPicker embedded with honesty rules; `min_shot_count` surfaced.
184     - [ ] `/api/capabilities` + `/api/health` + banner truthful; reload-survival via
`/api/videos` + `/api/reels`.
185     - [ ] **Observability:** correlation id threaded end-to-end; structured logs at every
new path; loud failures; logs inspected during CUJ replay.
186     - [ ] Retention sweep in place; `GEMINI_API_KEY` rotated; `allowed_video_root` set.
187     - [ ] CI green (L2 contract + L4 regression); coverage ? floor; $9 tracker all
??-resolved.
188
189     ## 12. References
190     **Engine code `[verified this session]`:**
`backend/main.py:12-17,42,52,307-328,331-340,342-396,638`;
`backend/api/uploads.py:64,89,123,147`; `backend/api/job_worker.py:37`;
`backend/api/static/app.js:88,344,373`;
`backend/storage/{base.py,local.py:33,ref.py,__init__.py:43}`;
`backend/config/models.py:129,183,298`;
`backend/pipeline/segmenters/{trajectory_hybrid.py:57-63,76-85, fusion_hybrid.py:76-86,
gemini.py:470-485, motion.py:191-198}`; `backend/pipeline/detectors/{native_wasb_runner.py,
stub_runner.py}`; `backend/worker/__main__.py:88,98,149`; `config.json` (`default_segmenter:
gemini`). **Engine docs:** `HOSTING_PLAN.md:42,102-108`; `VIDEO_LOCALITY_MODEL.md`;
`DESKTOP_APP_PLAN.md`; `SETUP_NEW_MACHINE.md`; `PLATFORM_ARCHITECTURE.md`;
`DECOUPLED_COMPUTE/{README,02,04,07,HANDOFF}.md`; `UI_PROPOSAL.md:41-66`;
`PROJECT_DOC_TEMPLATE.md`. **This repo:** `PER_RALLY_SELECTION.md`, `REQUEST_ACCESS_FORM.md`,
`PORTABILITY.md`, `web/README.md`.
191
192     ---
193     ### Changelog
194     - 2026-06-20 ? Initial `DRAFT`. Consolidated three single-box-appliance designs
(FastAPI-native / unified-origin / decoupled-profile) into the reverse-proxied unified-origin
spine + compute-as-a-setting framing, via a multi-agent design pass; every engine-fact anchor
re-verified against source before writing (catching the native-WASB partial-wiring +
`ai_max_workers` default nuances). Folded in the owner's ingest-menu (Drive/bucket/local),
multi-provider GPU, and first-class Observability directives.
195

```

17. assistant (2026-06-21T13:52:40.029Z)

I now have strong grounding. Let me check the WASB-SBDT upstream to confirm the torch version requirement and the DESKTOP_APP_PLAN + PORTABILITY docs to understand what's already been considered. Then I'll do the web research.

18. user (2026-06-21T13:52:40.991Z)

```

1     # Portability ? Cloudflare now, always-on VM if needed
2
3     > **Status:** design + proven (2026-06-20). **Decision:** keep the site/form host-
portable so
4     > moving off Cloudflare to an owner-run always-on VM ("if Cloudflare acts out") is an
**adapter
5     > swap, not a rewrite.**
6
7     ## The shape that makes it portable
8     - **Logic is runtime-agnostic.** `src/index.js` uses Web-standard `Request`/`Response`
and pure
9     helpers (`validate`, `toRow`, `isBot`, `handleRequest`) ? no host-specific calls.

```



```

10     - **All DB coupling is in adapters** (`src/store.js`): `d1Store` (Cloudflare D1) and
`sqliteStore`
11     (node:sqlite / better-sqlite3). `handleRequest(request, store)` takes the store by
injection.
12     - **One schema, one SQL.** `schema.sql` and the `INSERT` are plain SQLite, so D1 and a
VM's SQLite
13     file are byte-for-byte compatible ? zero dialect change. (Postgres later = minor
placeholder tweaks.)
14
15     ## Two hosts, same code
16     | | Cloudflare (now) | Always-on VM (fallback) |
17     |---|---|---|
18     | Entry | `src/index.js` (Worker `fetch`) | `server.js` (node:http) |
19     | Static | `env.ASSETS` | `server.js` serves `public/` |
20     | Storage | `d1Store(env.DB)` ? D1 | `sqliteStore(db)` ? node:sqlite file |
21     | Country | `cf-ipcountry` header | optional (GeoIP / null) |
22     | Deploy | git ? Workers Builds | `node server.js` under systemd |
23     | TLS/DNS | Cloudflare | keep Cloudflare in front, **or** Caddy/Let's Encrypt |
24
25     **Enforced, not asserted.** `node --test src/` runs BOTH paths and CI
(`.github/workflows/test.yml`,
26     Node 24) runs them on every PR, so the repo stays compatible with both hosts:
27     - `src/index.test.js` ? the Cloudflare Worker + D1 logic.
28     - `src/server.test.js` ? **boots the real `server.js`** (node:http + node:sqlite) and
round-trips it
29     over HTTP into SQLite (the same proof we run on the droplet, automated).
30
31     For a live check against a real VM: `SSH_TARGET=root@host scripts/droplet-smoke.sh` (scp
+ boot +
32     round-trip + cleanup). Verified live on the owner's droplet (Ubuntu 24.04, isolated
`/srv/khelsutra-site`).
33
34     ## Provider-portable provisioning ? destroy ? resume anywhere (scripted + TESTED)
35     Spinning up a fresh VM on **any** provider (DigitalOcean / GCE / any Ubuntu box) and
tearing it down
36     without losing data is a **first-class, scripted** flow ? so destroying a box and
resuming elsewhere is
37     routine, not risky.
38
39     - **`scripts/provision-vm.sh`** ? run from your LOCAL machine against a FRESH Ubuntu VM.
Pushes `site/`
40     via `git archive` (**private-repo-safe** ? no clone, token, or secret on the VM),
installs Node (?24,
41     so `node:sqlite` is unflagged), installs + starts a **systemd** service, opens the
firewall, and
42     **verifies** it serves (form + a real POST round-trip). Idempotent.
43     ...
44     SSH_TARGET=root@<ip> [SSH_KEY=~/.ssh/khelsutra_droplet] [PORT=8787]
[RESTORE_DB=./backup.db] \
45     site/scripts/provision-vm.sh
46     ...
47     - **`scripts/teardown-vm.sh`** ? **exports the SQLite state DB to a local timestamped
backup BEFORE**
48     anything is destroyed (registrations survive the move), stops the service, and
optionally destroys the
49     DO droplet (`DESTROY=1 DROPLET_ID=<id>`, via `doctl`).
50     ...
51     SSH_TARGET=root@<ip> [BACKUP_DIR=./vm-state-backups] [DESTROY=1 DROPLET_ID=<id>] \
52     site/scripts/teardown-vm.sh
53     ...
54     - **Clean resume on a different provider:** `teardown-vm.sh` (export) ? destroy ?
`RESTORE_DB=<backup>`
55     SSH_TARGET=root@<newip> site/scripts/provision-vm.sh`. The restored DB is `scp`'d in
before the service starts.
56

```

```

57      **Verified live (2026-06-20):** a fresh DO Ubuntu-24.04 droplet went bare ? operational
+ publicly
58      serving (HTTP 200) via `provision-vm.sh` (Node 24 auto-installed, systemd unit,
firewall); `teardown-vm.sh`
59      exported the state DB (12 KB); a re-provision with `RESTORE_DB` preserved the
registration (`rows:2` =
60      restored + the verify POST) ? then the throwaway droplet was destroyed. The
destroy?resume cycle is
61      reproducible across providers.
62
63      ## VM deploy runbook ("if Cloudflare acts out")
64      Node 22+ only ? **no `npm install`** (node:sqlite is built in).
65      ```
66      # on the VM:
67      git clone ? && cd khelsutra/site          # or scp the site/ folder
68      PORT=8080 DB_PATH=/var/lib/khelsutra/access.db node server.js
69      # put it under systemd (auto-restart) + a reverse proxy (Caddy/nginx) for TLS,
70      # then point DNS at the VM. ~1 hour; logic + schema unchanged.
71      ```
72
73      ## Honest notes
74      - "Cloudflare acts out" usually means **Workers/D1 pricing or limits, not DNS** ? so the
cheapest
75      move is to keep Cloudflare DNS/CDN in front and shift only the **compute** to the VM
(point a
76      route/origin at the box). Full exit (own DNS + Caddy TLS) is also supported.
77      - You can run **both** (Cloudflare primary + VM warm standby) since the schema is
identical; a
78      periodic `access_requests` export keeps them in sync.
79      - `server.js` is intentionally dependency-free; if you later want Postgres or a
framework (Hono runs
80      on Workers *and* Node), it slots in behind the same `store` + handler.
81

```

19. user (2026-06-21T13:52:43.713Z)

```

47:- **Apple Silicon has no CUDA.** WASB/TrackNet weights are PyTorch; PyTorch MPS (Metal)
backend *may* run them, but op-coverage and speed on the WASB model are **unverified**
`[design]`. CPU fallback on Apple is also unverified.
48:- **Owned-model tie-in.** `OWNED_MODEL_IMPLEMENTATION_PLAN.md` already states the on-device
answer: *train in Python ? **export across the boundary** (ONNX / Core ML / ExecuTorch / GGUF) ?
a thin device shell runs the artifact with zero Python*, with the standing constraint to **train
with export-clean ops**. A smaller, exported, quantized shuttle detector is a credible answer if
the CPU/MPS numbers on the current WASB weights are bad. **This makes P0 partly a referendum on
whether the desktop product needs the owned-model track.**
51:The detector layer was deliberately built to make this port low-regret.
[`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) documents,
verbatim, an **"Optional Linux-native path (the main de-risk goal)"**: a
`LinuxNativeRunner`/`ONNXRunner` that **"Skip[s] all wsl / _stage / conda activate, load[s]
weights locally (torch or onnx)? write[s] the exact same CSV format so
`trajectory_to_action_windows` works verbatim."** The WSL plumbing lives in a separate
`WslCommandMixin`, explicitly so a native runner **"must not inherit WSL plumbing."** The actual
inference math is already isolated in
[`wasb_infer.py`](../backend/pipeline/detectors/wasb_infer.py) (torch + the WASB-SBDT modules,
emitting `Frame,Visibility,X,Y`). **So the port is: lift `wasb_infer.py` out of WSL, load
weights with native torch/onnxruntime per-OS, keep the identical CSV contract.** The windowing
brain (`trajectory_to_action_windows`) and stitch are untouched.
56:| CUDA-native | Windows-native, Linux+NVIDIA | torch+CUDA (no WSL) | proven in WSL
`[verified]`; native = port |
57:| Apple MPS | macOS (Apple Silicon) | torch MPS / Core ML export | **unverified ? P0 must
measure** |
58:| CPU | all (the no-GPU laptop) | torch-CPU / **onnxruntime** (often fastest on CPU) |
**unverified ? P0 must measure; the gating number** |
59:| Smaller/exported model | all | ONNX/Core ML/ExecuTorch, quantized | deferred to owned-model
track; P0 informs whether it's *needed* |

```

```

130:| **P0** | **Compute-feasibility spike (make-or-break).** Measure WASB throughput for a
~10-min clip on: (a) CPU-only (torch-CPU & onnxruntime), (b) Apple MPS, (c) confirm CUDA-native
(no WSL). Output: a wall-clock table + a go/no-go verdict + "do we need a smaller model?"
recommendation. **No app code.** | D0 | **Owner greenlight** | ? | ? |
156:**Internal code anchors `[verified]`:**
[tools/generate_highlights.py](../tools/generate_highlights.py) (local orchestration; `--skip-ai-handoff`, `--wasb-stream`, `--detect-from`, `_WSL_DETECTORS`, `_atomic_publish`) .
[backend/pipeline/detectors/base.py](../backend/pipeline/detectors/base.py) (`DetectorRunner`
ABC + the documented Linux-native de-risk path + `WslCommandMixin`) .
[backend/pipeline/detectors/wasb_runner.py](../backend/pipeline/detectors/wasb_runner.py) (the
WSL adapter to replace) .
[backend/pipeline/detectors/wasb_infer.py](../backend/pipeline/detectors/wasb_infer.py) (the
torch inference core to lift native) . [backend/pipeline/detectors/tracknet_runner.py](../back
end/pipeline/detectors/tracknet_runner.py) (shared `trajectory_to_action_windows`) .
[backend/pipeline/segmenters/wasb_hybrid.py](../backend/pipeline/segmenters/wasb_hybrid.py) .
`backend/pipeline/stitcher/compiler.py::VideoCompiler` . `pyproject.toml` + `indexer` console
entry (#167).
159:**External (to validate at P0/P2/P3):** PyTorch MPS backend . onnxruntime
(CPU/CoreML/DirectML execution providers) . Tauri vs Electron sizing . ffmpeg LGPL build /
openh264 . Apple Developer-ID notarization . Windows Authenticode/MSIX.
===DESKTOP HEAD===

```

Cross-Platform Desktop App ? "Plug in your card, get your highlights"

```

> **Status:** `DRAFT` ? `owner-gated` (scoping doc only; **nothing is built**) . **Owner:**
avidullu . **Created:** 2026-06-18 . **Last updated:** 2026-06-18
> **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when DONE)
> **Tracking anchors:** $7 progress tracker is the source of truth; indexed in `docs/README.md`;
pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once the owner greenlights P0).
> **Relation to existing docs:** **peer-of**
[PLATFORM_ARCHITECTURE.md](PLATFORM_ARCHITECTURE.md) (this is the **`LocalDesktop`
ComputeBackend** made into a product) . **realizes the L0 tier** of
[VIDEO_LOCALITY_MODEL.md](VIDEO_LOCALITY_MODEL.md) . **consumes**
[OWNED_MODEL_IMPLEMENTATION_PLAN.md](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (the smaller/export-
clean model is one answer to the compute crux) . bound by
[COMMERCIALIZATION.md](COMMERCIALIZATION.md) licensing guardrails.
> **Honesty note:** every claim is tagged `[verified]` (checked against code/measured),
`[researched]` (needs sign-off), or `[design]` (proposed). **The entire build is `[design]`** ?
this doc scopes; it does not ship. Not legal advice.

```

0. TL;DR

Package the mature local pipeline (WASB shuttle detector + improved windowing + ffmpeg stitch, already runnable fully-offline via [tools/generate_highlights.py --skip-ai-handoff](../tools/generate_highlights.py)) as a ****cross-platform desktop app**** for ****non-technical recreational players****: plug in the camera's USB/SD card, click once, get back ****only the highlight reels**** ? fully on-device, ****no upload****, ****no copy of the GBs of 4K originals****. This is the product surface that fits the "local, cheap, efficient" thesis and the L0 ("nothing leaves the machine") tier of `VIDEO\_LOCALITY\_MODEL.md`.

****The make-or-break unknown is compute on a typical enthusiast laptop**** ? which often has ****no discrete GPU****. WASB runs at ****~3.4x real-time on a laptop RTX 2070 Super**** `[verified, measured]`; on CPU it is far slower, and Apple Silicon has ****no CUDA**** at all. ****P0 is a feasibility spike that measures CPU and Apple-MPS throughput for a ~10-min clip before any app is built.**** If CPU-only is unacceptable on commodity hardware, the path needs a smaller/faster owned model (ties `OWNED\_MODEL\_IMPLEMENTATION\_PLAN`) or a tiered "fast-if-GPU, slow-but-works-if-CPU" design. ****Everything past P0 is gated on that result.****

1. Problem & goal

****Why now / the user.**** Target users are ****recreational sports enthusiasts, not tech-savvy****. Their match videos are ****huge**** (GBs of 4K). The cloud-upload product surface fails them on every axis: slow on a typical Indian uplink (`VIDEO\_LOCALITY\_MODEL.md` \$0: 10 GB ? ~1 h at 20

Mbps, ~4.5 h at 5 Mbps), bandwidth-costly, privacy-fraught, and it bloats both our storage and their drives. The thing the customer actually wants back is **kilobytes of timestamps** ? a highlight reel (`VIDEO_LOCALITY_MODEL.md` $1: "timestamps are the product; the video is dead weight"`).

The concrete outcome ("good" looks like this):

1. User plugs the GoPro/phone SD card or camera USB into their laptop.
2. The app detects the card, lists the match videos on it.
3. User clicks **Generate Highlights**.
4. The app makes an on-device proxy, runs WASB detection + windowing + stitches the reel ? **100% locally**.
5. The reel(s) land in a user-chosen folder. **The originals are never copied off the card and never bloat any drive.**

No account, no upload, no cloud dependency in the default path. Gemini stays an **optional** "better brain if you want it" toggle ? never required, never bundled as a hard dependency, never a training source (`COMMERCIALIZATION` C4`).

Read to get current: `VIDEO_LOCALITY_MODEL.md`` (the L0/locality framing + OQ5 packaging seed), `PLATFORM_ARCHITECTURE.md` $3.2` (the ComputeBackend seam), `[`backend/pipeline/detectors/base.py](../backend/pipeline/detectors/base.py)` (the detector ABC that already anticipates a Linux-native runner), `tools/generate_highlights.py`` (the local entrypoint this app wraps).

2. Decisions locked

#	Decision	Source / date	Implication
D1	Default path is 100% on-device, no upload, no Gemini. The reel is produced from the local pipeline (<code>--skip-ai-handoff`</code> ). This doc · 2026-06-18; matches <code>VIDEO_LOCALITY_MODEL` L0</code> The app must never require a network call. Gemini is an opt-in toggle only.		
D2	Never copy the originals. The app reads source video in place (off the card / a chosen folder) and writes only reels + a tiny report to a user folder. This doc; mirrors <code>generate_highlights.py`</code> non-destructive <code>_atomic_publish`</code> discipline No "import library" that duplicates 10 GB files. Detection runs on a proxy; stitch reads the original in place.		
D3	Drop the WSL2 dependency; run the detector natively per-OS. This doc; enabled by the <code>DetectorRunner` ABC` [verified]`</code> Windows-native (no WSL), Linux+NVIDIA (CUDA directly), macOS (MPS or CPU). The WSL adapter stays for the current dev box but is not the shipped path.		
D4	Commercial-clean licensing for everything bundled: MIT / Apache-2.0 / BSD only (code, weights). ffmpeg ships as an LGPL build (no GPL-only encoders like x264). <code>COMMERCIALIZATION`</code> (ratified 2026-06-09); <code>VIDEO_LOCALITY_MODEL` OQ5` ("ffmpeg LGPL build = licensing-clean")</code>   WASB-SBDT weights are MIT <code>[verified]`</code> (see <code>tracknet-wasb-wsl2-decision`</code> ). The stitch must use openh264 (BSD) or OS hardware encoders, <b>not</b> statically-linked x264. See <code>\$3.4`</code> .		
D5	Reuse the existing pipeline verbatim; add only the shell + a native runner + device-IO. <code>backend/pipeline/detectors/base.py`</code> reuse contract <code>[verified]`</code> No re-architecture of windowing/stitch. New code = native runner, USB/SD detection, app shell, packaging.		
D6	Feasibility is gated on P0. No app shell, no packaging work starts until the compute spike returns a verdict. This doc; <code>decision-rigor-norm`</code> (NUMBERS INFORM, FOUNDATIONS DECIDE) P0 is the only un-gated deliverable; P1?P3 are owner-greenlit after P0 numbers.		

3. Foundation ? research / prior art `[design]` / [researched]``

3.1 The compute reality (the crux) ? what we know vs must measure

- **WASB on a discrete laptop GPU is acceptable** `[verified, measured]``: on the owner's **RTX 2070 Super Max-Q** (8 GB), WASB inference is the pipeline driver at **~3.2?3.4× real-time** (3.4 GPU-min per footage-min, ~0.055 s/frame), peak VRAM ~5.7?6.0 GB; proxy (NVENC) ~0.2× RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ?5× clip length. Source: `current-state.md`` `COST_SCALING.md`` checkpoint (6 Boxhill clips, 2026-06-18).
- **The typical enthusiast laptop has NO discrete GPU.** A 10-min 4K clip at 3.4× RT is ~34 GPU-min **with** a 2070S. On integrated graphics / CPU-only the wall-clock is unknown and could be 10×+ worse ? possibly **hours per clip**, which would kill the one-click promise.
- **Apple Silicon has no CUDA.** WASB/TrackNet weights are PyTorch; PyTorch MPS (Metal) backend

may run them, but op-coverage and speed on the WASB model are ****unverified**** `[design]`. CPU fallback on Apple is also unverified.

- ****Owned-model tie-in.**** `OWNED\_MODEL\_IMPLEMENTATION\_PLAN.md` already states the on-device answer: ***train in Python ? **export across the boundary** (ONNX / Core ML / ExecuTorch / GGUF) ? a thin device shell runs the artifact with zero Python*, with the standing constraint to **train with export-clean ops**.** A smaller, exported, quantized shuttle detector is a credible answer if the CPU/MPS numbers on the current WASB weights are bad. ****This makes P0 partly a referendum on whether the desktop product needs the owned-model track.****

3.2 The de-WSL native runner is already designed-for `[verified]`

20. assistant (2026-06-21T13:52:47.903Z)

Excellent context. The DESKTOP_APP_PLAN already flags an important angle: WASB is just torch + WASB-SBDT modules, and there's an explicit consideration of porting to ONNX. This is highly relevant to option (d). Now let me verify the actual WASB-SBDT torch version requirement and do the web research on packaging options and conda licensing.

21. user (2026-06-21T13:52:57.547Z)

Web search results for query: "WASB-SBDT nttcom shuttle detection requirements torch version pytorch 1.11"

Links: [{"title": "WASB-SBDT/GET_STARTED.md at main · nttcom/WASB-SBDT", "url": "https://github.com/nttcom/WASB-SBDT/blob/main/GET_STARTED.md"}, {"title": "GitHub - nttcom/WASB-SBDT: [BMVC2023] Widely Applicable Strong Baseline for Sports Ball Detection and Tracking · GitHub", "url": "https://github.com/nttcom/WASB-SBDT"}, {"title": "PyTorch Release 21.11 - NVIDIA Docs", "url": "https://docs.nvidia.com/deeplearning/frameworks/pytorch-release-notes/rel_21-11.html"}, {"title": "Efficient Semantic Segmentation on Edge Devices", "url": "https://arxiv.org/pdf/2212.13691"}, {"title": "Fine-Tuning Deteriorates General Textual Out-of-Distribution Detection by Distorting Task-Agnostic Features", "url": "https://arxiv.org/pdf/2301.12715"}, {"title": "TorchLens: A Python package for extracting and visualizing hidden activations of PyTorch models", "url": "https://www.biorxiv.org/content/10.1101/2023.03.16.532916.full.pdf"}, {"title": "TorchLens: A Python package for extracting and visualizing hidden activations of PyTorch models", "url": "https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10055035/"}]

Based on the search results, I found information about the WASB-SBDT project from NTT Communications. WASB (Widely Applicable Strong Baseline) is a Sports Ball Detection and Tracking baseline that can be applied to a wide range of sports categories.

Regarding the PyTorch version requirements, the search results show that the project has been tested with Python 3.8 and CUDA 11.3 on Ubuntu 18.04, but the specific PyTorch 1.11 requirement you mentioned in your query is not explicitly stated in these results.

The search results do not contain detailed information about specific "shuttle detection requirements" or confirmation of PyTorch 1.11 compatibility. For more detailed requirements, you may want to check the repository's `requirements.txt` file or the complete README documentation on the [WASB-SBDT GitHub repository](https://github.com/nttcom/WASB-SBDT).

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

22. user (2026-06-21T13:53:02.978Z)

Web search results for query: "Anaconda Terms of Service commercial use 2024 2025 paid required organization 200 employees"

Links: [{"title": "Terms of Service | Anaconda", "url": "https://www.anaconda.com/legal/terms/terms-of-service"}, {"title": "Anaconda Legal | Anaconda", "url": "https://www.anaconda.com/legal"}, {"title": "Update on Anaconda's Terms of Service for Academia and Research | Anaconda", "url": "https://www.anaconda.com/blog/update-anacondas-terms-of-service-for-academia-and-research"}, {"title": "Retrospective on Anaconda's"}]

2024 Licensing Changes: What They Mean and Smarter Alternatives", "url": "https://licenseware.io/retrospective-on-anacondas-2024-licensing-changes-what-they-mean-and-smarter-alternatives/"}], [{"title": "Anaconda's terms of service", "url": "https://legal.anaconda.com/policies/en/?name=terms-of-service"}], [{"title": "Navigating Anaconda Licensing Changes: What You Need to Know | DataCamp", "url": "https://www.datacamp.com/blog/navigating-anaconda-licensing"}], [{"title": "Anaconda Transition Working Group ? Campus Research Computing Consortium (CaRCC)", "url": "https://carcc.org/anaconda-transition-working-group/"}], [{"title": "Anaconda Licensing: What Changes with the New ...", "url": "https://www.linkedin.com/pulse/anaconda-licensing-what-changes-new-license-terms-x5jaf"}], [{"title": "New Anaconda Terms Of Service (TOS) - Issue #3 · robocorp/rcc", "url": "https://github.com/robocorp/rcc/issues/3"}]]

Based on the search results, here's what I found regarding Anaconda's Terms of Service regarding commercial use, paid licenses, and the 200-employee threshold:

Commercial Use Requirements

Any organization with 200 or more employees or contractors is now required to purchase a paid license to use Anaconda's software. The 200-employee threshold applies not only to for-profit companies but also to government entities and non-profit organizations.

When a Paid License is Needed

The distribution of packages Anaconda builds and maintains remains free for individuals and small organizations, which is defined in the Terms of Service as having less than 200 employees or contractors. A paid license is required for large organizations, which is defined in the Terms of Service as having more than 200 or employees or contractors.

Academic Exemptions

Anaconda is free for all educational entities?even those with 200 or more employees or contractors?when used in a course curriculum, including teaching, learning, and researching, at accredited universities worldwide.

Recent Changes (2024-2025)

Starting in March 2024, Anaconda updated its Terms of Service to introduce a new Commercial Edition License. Under this license, any organization with 200 or more employees?whether private, public, governmental, or even non-profit, is now required to purchase a commercial license to legally use Anaconda's software.

Effective July 15, 2025, Users who download and use the newest release of Miniconda will be prompted with a notice that accessing the Anaconda Repositories requires acceptance of our Terms of Service via the command line.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

23. assistant (2026-06-21T13:53:07.352Z)

Important nuance: the Anaconda paid-license threshold is 200 employees (so for a single founder it's free), BUT the bigger trap is bundling/redistributing the Anaconda `defaults` channel in a downloadable app. Let me verify the WASB-SBDT exact torch requirement and the conda-forge/miniforge vs Anaconda channel distinction (which is the real licensing trap for a redistributable app).

24. user (2026-06-21T13:53:11.898Z)

Based on the provided GET_STARTED.md content, I cannot find specific installation requirements detailing Python version, PyTorch/torch version, torchvision version, or CUDA version.

The document only mentions:

- Running setup scripts via `sh setup\_scripts/setup\_soccer.sh`
- Executing evaluation commands like `python3 main.py`

However, the exact dependency specifications, environment creation method (conda/pip), and version numbers are not included in this document. You would need to consult additional files in the repository, such as a `requirements.txt`, `setup.py`, `environment.yml`, or `README.md` file, to find these technical specifications.

25. user (2026-06-21T13:53:15.416Z)

Web search results for query: "miniforge conda-forge channel license redistribution commercial free vs Anaconda defaults channel BSD"

Links: [{"title": "Is Conda actually free? | pydevtools", "url": "https://pydevtools.com/handbook/explanation/is-conda-actually-free/"}, {"title": "Miniforge: The minimal, open solution for institutional Python environments - Fabrizio Musacchio", "url": "https://www.fabriziomusacchio.com/blog/2025-07-03-miniforge/"}, {"title": "Getting Started with Conda at HPCC | User Guides | High Performance Computing Center | Texas Tech", "url": "https://www.depts.ttu.edu/hpcc/userguides/application_guides/Miniforge.php"}, {"title": "Handling Anaconda without getting Constricted - Florian Wilhelm's blog", "url": "https://florianwilhelm.info/2021/09/Handling_Anaconda_without_getting_constricted/"}, {"title": "A Guide to the Anaconda defaults channel | ubinfie Blog", "url": "https://ubinfie.github.io/2024/10/15/anaconda-defaults.html"}, {"title": "miniforge/LICENSE at main · conda-forge/miniforge", "url": "https://github.com/conda-forge/miniforge/blob/main/LICENSE"}, {"title": "GitHub - conda-forge/miniforge: A conda-forge distribution. · GitHub", "url": "https://github.com/conda-forge/miniforge"}]

Based on the search results, here's a comprehensive overview of the licensing and distribution differences:

Miniforge vs Anaconda Defaults Channel

Miniforge with conda-forge:

Miniforge ships conda, mamba, and Python with conda-forge as the default channel with no commercial license required. Miniforge is distributed under a permissive BSD/MIT license and is entirely independent of Anaconda, Inc. Furthermore, Miniforge installer code uses BSD-3-Clause license.

The conda-forge channel is community-maintained, hosts over 25,000 packages, and is free for all use, including commercial.

Anaconda Defaults Channel Licensing:

In contrast, Anaconda, Inc. operates a curated package repository at repo.anaconda.com, and accessing that repository requires a paid license for any organization with more than 200 employees or contractors.

Key Differences

Organizations are recommended to transition to MiniForge to avoid potentially restrictive terms of service, and MiniForge starts with the community-supported and free conda-forge channel without the complexities of commercial licensing.

Conda-forge has no restrictions for commercial activities, so you can remove the defaults channel from your conda configuration and use only the conda-forge repository instead to be legally on the safe side.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

26. assistant (2026-06-21T13:53:19.165Z)

Key licensing distinction confirmed: the trap is the `**`defaults` channel**` (repo.anaconda.com), not conda the tool. conda-forge / miniforge / micromamba are BSD-3-Clause and free for commercial use including redistribution. Let me fetch the WASB-SBDT requirements directly and research micromamba/conda-pack for self-contained env packaging.

27. user (2026-06-21T13:53:20.753Z)

The server returned HTTP 404 Not Found.

The response body was not retrieved. If this URL requires authentication, use an authenticated tool (e.g. ``gh`` for GitHub, or an MCP-provided fetch tool) instead of WebFetch.

28. user (2026-06-21T13:53:27.479Z)

Web search results for query: "micromamba self-contained environment bundle redistribute application conda-pack relocatable prefix"

Links: [{"title": "Micromamba Installation - Mamba's documentation!", "url": "https://mamba.readthedocs.io/en/latest/installation/micromamba-installation.html"}, {"title": "Micromamba Usage Guide | Research IT", "url": "https://research.it.iastate.edu/micromamba-usage-guide"}, {"title": "Making mamba self-contained/Having user configured ~/.conda and ~/.condarc alternatives · Issue #3123 · mamba-org/mamba", "url": "https://github.com/mamba-org/mamba/issues/3123"}, {"title": "Conda - ULHPC Technical Documentation", "url": "https://hpc-docs.uni.lu/environment/conda/"}, {"title": "GitHub - mamba-org/mamba: The Fast Cross-Platform Package Manager · GitHub", "url": "https://github.com/mamba-org/mamba"}, {"title": "Micromamba User Guide ? documentat ion", "url": "https://mamba.readthedocs.io/en/stable/user_guide/micromamba.html"}, {"title": "Using Mamba to install Conda packages", "url": "https://wiki.unil.ch/ci/books/high-performance-computing-hpc/page/using-mamba-to-install-conda-packages"}, {"title": "Micromamba Installation ? documentation", "url": "https://mamba.readthedocs.io/en/stable/installation/micromamba-installation.html"}, {"title": "Conda to micromamba in dockerfile - JupyterHub - Jupyter Community Forum", "url": "https://discourse.jupyter.org/t/conda-to-micromamba-in-dockerfile/25452"}]

I've completed the web search for your query. Here are the key findings:

Search Results Summary

The search results provide information about micromamba, a self-contained package manager, along with related tools and concepts:

Micromamba as Self-Contained Executable

Micromamba is a fully statically-linked, self-contained executable, which means the base environment is completely empty. Micromamba is the statically linked version of mamba and can be installed as a standalone executable without any dependencies, making it a perfect fit for CI/CD pipelines and containerized environments.

Configuration & Relocatable Prefix

All environments and cache will be created by default under the `MAMBA_ROOT_PREFIX` environment variable. This allows for relocatable prefix configurations, enabling the environment to be moved or redistributed while maintaining functionality.

Environment Management

The search results show that micromamba supports creating and managing isolated environments, which can be bundled and redistributed. Micromamba can be used to install lock files generated by conda-lock without having to install conda-lock, by invoking micromamba create with the `-f` option.

Self-Contained Bundle Use Cases

Scenarios where relying on a single self-contained executable is required are well-suited for

micromamba deployment.

Note: The search results mention conda-pack and relocatable environments conceptually, but don't provide detailed technical documentation on those specific tools in relation to micromamba bundling.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

29. assistant (2026-06-21T13:53:30.626Z)

Let me get the actual WASB-SBDT requirements from GitHub via the gh CLI (authenticated), and verify whether torch 1.x is truly required or whether it runs on modern torch.

30. user (2026-06-21T13:53:35.018Z)

```
REQ_FAILED
=== ENV_YML ===
NO_ENV_YML
=== ROOT LISTING ===
.gitignore
Dockerfile
GET_STARTED.md
LICENSE.md
MODEL_ZOO.md
README.md
src
```

31. assistant (2026-06-21T13:53:38.956Z)

There's a Dockerfile. Let me read it and GET_STARTED.md directly via gh ? the Dockerfile will tell me the exact torch/CUDA base image.

32. user (2026-06-21T13:53:42.488Z)

```
=== Dockerfile ===
FROM nvcr.io/nvidia/tensorrt:21.06-py3

MAINTAINER shuheitarashima@ntt.com
LABEL OBJECT="Dockerfile for Widely Applicable Strong Baseline for Sports Ball Detection and Tracking [BMVC2023]"
ARG http_proxy
ARG https_proxy
ENV http_proxy ${http_proxy}
ENV https_proxy ${https_proxy}
ENV DEBIAN_FRONTEND noninteractive

RUN apt-get update && apt-get install -y --no-install-recommends \
    apt-utils \
    apt-transport-https \
    ca-certificates \
    software-properties-common \
&& apt-get clean \
&& rm -rf /var/lib/apt/lists/*
RUN apt-get update && apt-get install -y \
    git \
    vim \
    cmake \
    libsm6 \
    libxext6 \
    libxrender-dev \
    libgl1-mesa-glx \
    locales \
```

```

build-essential\
zlib1g-dev\
libssl-dev\
libbz2-dev\
libreadline-dev\
libsqlite3-dev\
libhdf5-dev\
libncursesw5-dev\
libgdbm-dev\
liblzma-dev\
libdb-dev\
libffi-dev\
uuid-dev\
tk-dev\
&& apt-get clean\
&& rm -rf /var/lib/apt/lists/*

ENV LANG en_US.UTF-8
ENV LANGUAGE en_US:en
ENV LC_ALL en_US.UTF-8
RUN locale-gen en_US.UTF-8

RUN pip3 install numpy==1.22.4 torch==1.11.0+cu113 torchvision==0.12.0+cu113 --extra-index-url
https://download.pytorch.org/whl/cu113
RUN pip3 install hydra-core==1.2.0 tqdm==4.64.0 opencv-python==4.6.0.66 scikit-learn==1.1.1
scikit-image==0.19.3 pandas==1.3.5 einops==0.4.1 timm==0.6.5 matplotlib==3.5.2

COPY src /root/src
WORKDIR /root/src
#COPY .vimrc /root/

=== GET_STARTED.md ===

```

Get Started

Data Preparation

Download [Soccer](https://pspagnolo.jimdo.free.com/download/), [Tennis](https://nol.cs.nctu.edu.tw:234/open-source/TrackNet), [Badminton](https://nol.cs.nctu.edu.tw:234/open-source/TrackNetv2), [Volleyball](https://github.com/mostafa-saad/deep-activity-rec), and [Basketball](https://ruiyan1995.github.io/SAM.html) and put them under ``<WASB-SBDT\_HOME>/datasets``.

- For Soccer, we provide a setup script. Run ``cd ./src && sh setup\_scripts/setup\_soccer.sh``.
- For Tennis, download ``Dataset.zip`` from [this link](https://nycu1-my.sharepoint.com/:u:/g/personal/tik_m365_nycu_edu_tw/ETCr6-M0e1VDhGCdMbvIjcsBu31AJT05xa_1cW8pHa7niA?e=55tLJ9) and put it under ``<WASB-SBDT\_HOME>`` directory. Then unzip the file and rename ``Dataset`` directory as ``tennis``.
- For Badminton, download ``TrackNetV2.zip`` from [this link](https://nycu1-my.sharepoint.com/:u:/g/personal/tik_m365_nycu_edu_tw/EWisYhAiai9Ju7L-tQp0yKEBZJd9VQkKqsFrjcqYIDP-g?e=S0AB1Z) and put it under ``<WASB-SBDT\_HOME>`` directory. Then use a setup script by running ``cd ./src && sh setup\_scripts/setup\_badminton.sh``.
- For Volleyball, download ``volleyball.zip`` from [this link](https://drive.google.com/drive/folders/1rmsrG1mgkwxOKhsr-QYoi9Ss92wQmCOS?usp=sharing) and ``volleyball\_ball\_annotations.zip`` from [this link](https://drive.google.com/file/d/1urZpZiiepC85JD1u3VeURgUpztRgI0yl/edit) and unzip them. Then put the resulting directories as follows.
- For Basketball, download all the zip segments shown [here](https://ruiyan1995.github.io/SAM.html) and unzip them to generate ``NBA\_data``. Then run a setup script i.e., ``cd ./src && sh setup\_scripts/setup\_basketball.sh``.

Data structure should be the following:

```

...
datasets
|-----soccer
|         ?-----videos
|         ?-----frames
|         ?-----annos
?-----tennis /* renamed from Dataset */
|         ?-----game1
|         ?-----...
|         ?-----game10
?-----badminton
|         ?-----match1
|         ?-----...
|         ?-----match26
|         ?-----test_match1
|         ?-----...
|         ?-----test_match3
?-----volleyball
|         ?-----videos
|         ?-----volleyball_ball_annotation
?-----basketball /* renamed from NBA_data */
|         ?-----videos
|         ?-----ball-annos
|
src
...

```

Model Preparation

Download pretrained models listed in [MODEL_ZOO.md](./MODEL_ZOO.md).

We also provide a setup script to download all the listed models at once. Run ``cd ./src && sh setup\_scripts/setup\_weights.sh``, then models are located in ``<WASB-SBDT\_HOME>/pretrained\_weights``.

Evaluation

Here we show the evaluation commands to reproduce the results of Table 2 and Table 3 in [our paper](https://arxiv.org/abs/2311.05237).

[WASB (Ours, Step=3)](https://arxiv.org/abs/2311.05237)

```
...
```

Soccer

```
python3 main.py --config-name=eval dataset=soccer model=wasb
detector.model_path=./pretrained_weights/wasb_soccer_best.pth.tar
```

Tennis

```
python3 main.py --config-name=eval dataset=tennis model=wasb
detector.model_path=./pretrained_weights/wasb_tennis_best.pth.tar
```

33. assistant (2026-06-21T13:53:47.570Z)

Now I have the exact pins: `numpy==1.22.4 torch==1.11.0+cu113 torchvision==0.12.0+cu113` plus `hydra-core==1.2.0 timm==0.6.5 einops==0.4.1` etc. The WASB model is a CNN (HRNet-style backbone via timm) with a simple tracker ? these are very standard ops. This strongly informs the port-to-torch-2.x and ONNX-export options. Let me verify whether torch 1.11 pins are truly load-bearing (do the WASB ops break on torch 2.x?) and research the ONNX export path for HRNet/CNN models, plus PyInstaller bundling of torch.

34. user (2026-06-21T13:53:55.414Z)

Web search results for query: "timm 0.6.5 HRNet load checkpoint torch 2.x compatibility breaking changes.pth.tar weights_only"

Links: [{"title": "timm-eidl - PyTorch Image Models", "url": "https://pypi.org/project/timm-eidl/"}, {"title": "timm 0.9.9", "url": "https://pypi.org/project/timm/0.9.9/"}, {"title": "timm 0.9.10", "url": "https://pypi.org/project/timm/0.9.10/"}, {"title": "Changelog · Hugging Face", "url": "https://huggingface.co/docs/timm/changes"}, {"title": "Pytorch Image Models (timm) | timmdocs", "url": "https://timm.fast.ai/"}, {"title": "timm 0.9.2", "url": "https://pypi.org/project/timm/0.9.2/"}, {"title": "timm · PyPI", "url": "https://pypi.org/project/timm/"}, {"title": "GitHub - rwrightman/timm: PyTorch image models, scripts, pretrained weights -- ResNet, ResNeXT, EfficientNet, EfficientNetV2, NFNet, Vision Transformer, MixNet, MobileNet-V3/V2, RegNet, DPN, CSPNet, and more · GitHub", "url": "https://github.com/rwrightman/timm"}]

Based on the search results, I found relevant information about timm and PyTorch 2.x compatibility. Here are the key findings:

Recent versions of timm have improved pickle checkpoint handling security and default all loading to `weights_only=True`, with support for `safe_global` for `ArgParse`. This is particularly important for PyTorch 2.x compatibility, as the `weights_only` parameter relates to how checkpoint files (like `.pth` and `.pth.tar` files) are loaded.

The search results indicate that timm 0.6.x releases maintain stability, and support for PyTorch 2.0 compile is available through the `--torchcompile` argument.

Regarding the specific issue of loading `.pth.tar` checkpoint files with timm 0.6.5 and PyTorch 2.x, the `weights_only=True` default is a breaking change in PyTorch 2.x that improves security by restricting pickle deserialization. If you're encountering issues loading HRNet checkpoints with this setting, you may need to either:

1. Convert your checkpoint to a compatible format
2. Use `weights_only=False` if you trust the checkpoint source (not recommended for untrusted sources)
3. Update to a newer version of timm that may have improved handling for this scenario

Unfortunately, the search results don't contain specific details about HRNet checkpoint compatibility issues in timm 0.6.5. For more targeted information, you might want to check the timm GitHub repository's issues or release notes for version 0.6.5 specifically.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

35. user (2026-06-21T13:53:56.225Z)

Web search results for query: "PyInstaller bundle pytorch CUDA application size downloadable app torch wheel large nuitka"

Links: [{"title": "Reducing the Size of PyTorch-Based Executable · pyinstaller · Discussion #8552", "url": "https://github.com/orgs/pyinstaller/discussions/8552"}, {"title": "Reducing the Size of PyTorch-Based Executable · Issue #8551 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/8551"}, {"title": "Simplifying Deployment: Using PyInstaller with PyTorch ? codegenes.net", "url": "https://www.codegenes.net/blog/pyinstaller-pytorch/"}, {"title": "Why torch wheel is so huge (582MB)? · Issue #17621 · pytorch/pytorch", "url": "https://github.com/pytorch/pytorch/issues/17621"}, {"title": "Pyinstaller & pytorch - deployment - PyTorch Forums", "url": "https://discuss.pytorch.org/t/pyinstaller-pytorch/185917"}, {"title": "PyInstaller, issue to build a package with CUDA, CuDNN and Tensorflow on Windows 10 · Issue #7175 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/7175"}]

Based on the search results, here are the key findings regarding bundling PyTorch with CUDA using PyInstaller:

Application Size Issues

PyTorch with CUDA (torch2.3+cu118) in conda is around 5GB, and the torch library in the bundled app is about 4GB. Real-world examples show that a simple app including PyTorch and PyQt using PyInstaller resulted in a 4GB dist folder.

Why the Size is Large

Most of the size consists of binary extensions and CUDA/cuDNN shared libraries, which cannot be split. Using torch while keeping the executable file small are mutually exclusive aspects, especially for CUDA-enabled torch builds.

Size Optimization Strategies

To reduce bundle size, you can:

- Remove unnecessary imports from your Python script and only import the modules and functions that you actually need
- Use the `--exclude-module` option to exclude certain modules not required, such as excluding CUDA-related modules if not needed

The Core Challenge

Applications using CPU-only torch builds (no CUDA) or older builds with smaller CUDA libraries are significantly smaller, but this requires compromising on CUDA acceleration. The fundamental constraint is that the PyTorch GPU wheel itself is approximately 583MB, and this grows substantially when additional CUDA dependencies are included.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

36. assistant (2026-06-21T13:54:00.843Z)

This confirms the size problem with bundling torch (4-5GB per env, ×2 envs = ~10GB). Now let me check the WASB model architecture to assess ONNX-export feasibility, and verify the conda-pack license and whether the WASB-SBDT LICENSE is MIT. Let me also check the engine's own torch version claim and the README/install docs.

37. user (2026-06-21T13:54:04.041Z)

```
=== WASB LICENSE ===  
MIT License
```

```
Copyright (c) 2023 NTT Communications Corporation
```

```
Permission is hereby granted, free of charge, to any person obtaining a copy  
of this software and associated documentation files (the "Software"), to deal  
in the Software without restriction, including without limitation the rights  
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell  
copies of the Software, and to permit persons to whom the Software is  
furnished to do so, subject to the following conditions:
```

```
The above copyright notice and this permission notice shall be included in all  
copies or substantial portions of the Software.
```

```
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR  
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,  
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE  
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER  
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,  
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE  
SOFTWARE.
```

```
=== src listing ===  
configs  
dataloaders
```

datasets
detectors
losses
main.py
models
optimizers
runners
setup_scripts
trackers
utils

38. user (2026-06-21T13:54:12.321Z)

Web search results for query: "conda-pack license BSD relocatable environment torch shared libraries pack for distribution caveats"

Links: [{"title": "GitHub - conda/conda-pack: Package conda environments for redistribution . GitHub", "url": "https://github.com/conda/conda-pack"}, {"title": "Conda-Pack ? conda-pack 0.8.1 documentation", "url": "https://conda.github.io/conda-pack/"}, {"title": "GitHub - jcrist/conda-pack: Package conda environments for redistribution . GitHub", "url": "https://github.com/jcrist/conda-pack"}, {"title": "conda-pack - anaconda | Anaconda.org", "url": "https://anaconda.org/anaconda/conda-pack"}, {"title": "Sbcast Conda Environments ? OLCF User Documentation", "url": "https://docs.olcf.ornl.gov/software/python/sbcast_conda.html"}, {"title": "conda-pack - conda-forge | Anaconda.org", "url": "https://anaconda.org/conda-forge/conda-pack"}, {"title": "GitHub - conda/conda: A system-level, binary package and environment manager running on all major operating systems and platforms. . GitHub", "url": "https://github.com/conda/conda"}, {"title": "Conda Pack :: Anaconda.org", "url": "https://anaconda.org/ctools/conda-pack"}, {"title": "Conda license ? conda-docs documentation", "url": "https://docs.conda.io/en/latest/license.html"}, {"title": "Using shared libraries ? conda-build 0.0.0.dev0+placeholder documentation", "url": "https://docs.conda.io/projects/conda-build/en/stable/resources/use-shared-libraries.html"}]

Based on the search results, here's what I found about conda-pack:

License

Conda-pack is offered under a New BSD license.

Purpose and Function

Conda-pack is a command line tool for creating relocatable conda environments. This is useful for deploying code in a consistent environment, potentially in locations where python or conda isn't already installed.

Key Caveats

The tool has several important limitations:

1. Conda must be installed and be on your path. The OS where the environment was built must match the OS of the target.
2. Once an environment is unpacked and conda-unpack has been executed, it cannot be relocated. Re-applying conda-pack is unlikely to work.
3. Conda-pack is not well-suited for archiving old environments, because it requires that conda's package cache have all of the environment's packages present. It is intended for building archives from actively maintained conda environments.

The search results also show that conda-pack can be used to package environments containing PyTorch and other libraries for distribution and relocation across different systems.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

39. assistant (2026-06-21T13:54:16.538Z)

I have everything I need. Let me do two more targeted searches: one on whether torch 1.11?2.x is achievable for this model class (HRNet/timm CNN with a simple tracker, no exotic ops), and one on the ONNX runtime cross-version benefit, to firm up options (d) and the recommendation.

40. user (2026-06-21T13:54:23.964Z)

Web search results for query: "pytorch 1.11 model run on torch 2.x backward compatible state_dict load CNN inference no breaking ops"

Links: [{"title": "Module ? PyTorch 2.11 documentation", "url": "https://docs.pytorch.org/docs/2.11/generated/torch.nn.Module.html"}, {"title": "Models and pre-trained weights ? Torchvision 0.12 documentation", "url": "https://docs.pytorch.org/vision/0.12/models"}, {"title": "A Comprehensive Guide to PyTorch for Image Processing ? codegenes.net", "url": "https://www.codegenes.net/blog/pytorch-image/"}, {"title": "PyTorch 2.x", "url": "https://pytorch.org/get-started/pytorch-2-x/"}, {"title": "Models and pre-trained weights ? Torchvision 0.27 documentation", "url": "https://docs.pytorch.org/vision/stable/models.html"}, {"title": "Models and pre-trained weights ? Torchvision main documentation", "url": "https://docs.pytorch.org/vision/main/models.html"}, {"title": "Save and Load Models in PyTorch - GeeksforGeeks", "url": "https://www.geeksforgeeks.org/deep-learning/save-and-load-models-in-pytorch/"}, {"title": "torch.nn ? PyTorch 0.1.12_2 documentation", "url": "https://docs.pytorch.org/docs/0.1.12/nn.html"}, {"title": "Building a Convolutional Neural Network (CNN) from Scratch with PyTorch | by Ugama Benedicta Kelechi | Medium", "url": "https://medium.com/@ugamakelechi501/building-a-convolutional-neural-network-cnn-from-scratch-with-pytorch-eca3ffdcf2ff"}, {"title": "Releases · pytorch/pytorch", "url": "https://github.com/pytorch/pytorch/releases"}]

Based on the web search results, I found relevant information about PyTorch backward compatibility and state_dict loading:

Key Findings:

****Backward Compatibility for state_dict:****

Backward compatibility is guaranteed for loading a serialized state_dict to the model created using old PyTorch version. However, loading entire saved models or serialized ScriptModules (serialized using older versions of PyTorch) may not preserve the historic behaviour.

This means you can save a state_dict from PyTorch 1.11 and load it into PyTorch 2.x by creating the model architecture in the new version and then using `load_state_dict()`.

****How to Load State Dict for CNN Inference:****

For CNN models specifically, the process involves:

1. Creating a new model instance with the same architecture in PyTorch 2.x
2. Using the `state_dict()` method to store the parameters of the created model, and when the model is loaded, a new model with the same architecture is created, then the parameters of the new model are replaced with the stored parameters.

****Important Note on C APIs:****

PyTorch will keep the set of C APIs stable across PyTorch versions and thus provide backward compatibility guarantees for AOTInductor-compiled models.

This indicates that standard tensor operations used in CNN inference should remain compatible across versions, with no breaking ops expected for typical CNN operations like convolutions, pooling, and ReLU activations.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

41. user (2026-06-21T13:54:26.098Z)

Web search results for query: "onnxruntime export pytorch CNN model run without torch dependency cross-platform CPU CUDA single file deployment"

Links: [{"title": "Export a PyTorch model to ONNX ? PyTorch Tutorials 2.12.0+cu130 documentation", "url": "https://docs.pytorch.org/tutorials/beginner/onnx/export_simple_model_to_onnx_tutorial.html"}, {"title": "Export PyTorch model | onnxruntime", "url": "https://onnxruntime.ai/docs/tutorials/export-pytorch-model.html"}, {"title": "Exporting PyTorch Models to ONNX", "url": "https://apxml.com/courses/advanced-pytorch/chapter-4-deployment-performance-optimization/exporting-onnx"}, {"title": "Exporting Mask R-CNN Models from PyTorch to ONNX ? Christian Mills", "url": "https://christianjmill.com/posts/pytorch-train-mask-rcnn-tutorial/onnx-export/"}, {"title": "Exporting Keypoint R-CNN Models from PyTorch to ONNX ? Christian Mills", "url": "https://christianjmill.com/posts/pytorch-train-keypoint-rcnn-tutorial/onnx-export/"}, {"title": "ONNX Runtime | Pytorch", "url": "https://onnxruntime.ai/pytorch"}, {"title": "Export PyTorch model | onnxruntime", "url": "https://faxu.github.io/onnxruntime/docs/tutorials/export-pytorch-model.html"}, {"title": "Deploy models into production (advanced) ? PyTorch Lightning 2.6.1 documentation", "url": "https://lightning.ai/docs/pytorch/stable/deploy/production_advanced.html"}, {"title": "Run PyTorch models on the edge", "url": "https://onnxruntime.ai/blogs/pytorch-on-the-edge"}, {"title": "PyTorch Inference | onnxruntime", "url": "https://onnxruntime.ai/docs/tutorials/accelerate-pytorch/pytorch.html"}]

Based on the search results, here's comprehensive information about exporting PyTorch CNN models to ONNX and running them with ONNX Runtime:

Exporting PyTorch CNN Models to ONNX

PyTorch includes support for ONNX through the `torch.onnx` APIs to simplify exporting your PyTorch model to the portable ONNX format. PyTorch provides the `torch.onnx.export()` function within the `torch.onnx` module as the primary tool for this conversion, which typically uses tracing to record the operations executed when a sample input passes through the model, converting these operations into their ONNX equivalents.

Key Benefits for Cross-Platform Deployment

ONNX is a flexible open standard format for representing machine learning models which standardized representations of machine learning allow them to be executed across a gamut of hardware platforms and runtime environments from large-scale cloud-based supercomputers to resource-constrained edge devices, such as your web browser and phone.

The first step for running PyTorch models on the edge is to get them into a lightweight format that doesn't require the PyTorch framework and its gigabytes of dependencies. The PyTorch ONNX APIs take the Pythonic PyTorch code and turn it into a functional graph that captures the operators that are needed to run the model without Python.

Running with ONNX Runtime

Exporting your PyTorch models to ONNX allows them to run on a wide variety of platforms and inference engines, such as ONNX Runtime, TensorRT, OpenVINO, and various mobile/edge devices, often benefiting from hardware-specific optimizations provided by these runtimes.

To execute inference on CPU and GPU: You can create an `InferenceSession` with different providers like `CPUExecutionProvider` for inference without PyTorch dependencies.

Deployment Advantages

ONNX Runtime was built on the experience of taking PyTorch models to production in high scale services like Microsoft Office, Bing, and Azure, and with ONNX Runtime, models can be ready to be deployed at scale in hours or days.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

42. assistant (2026-06-21T13:54:30.023Z)

I now have comprehensive grounding on all options. Let me do one final check on the engine's torch version claim and existing install docs to confirm the "main env = torch 2.6" detail and whether there's any prior ONNX work, then synthesize.

43. user (2026-06-21T13:54:32.037Z)

Found 2 files
backend\eval\ablation_pdf.py
backend\pipeline\detectors\base.py

44. user (2026-06-21T13:54:33.480Z)

```
=== requirements.txt ===  
fastapi>=0.111.0  
uvicorn>=0.30.1  
opencv-python>=4.9.0.80  
numpy>=1.26.4  
pydantic>=2.7.1  
jinja2>=3.1.4  
python-dotenv>=1.0.0  
google-genai>=2.4.0  
torch>=2.12.0  
torchvision>=0.27.0  
supervision>=0.21.0,<0.30.0
```

Per-video observability reports (SOFT dependency: the pipeline degrades to a no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once the private repo is published, e.g.:

obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3

For local development install it editable from a sibling checkout:

pip install -e ../sports-obsreport --no-build-isolation

```
=== ci torch line ===  
52: # Deliberately does NOT install cv2/torch/numpy ? the smoke tests must hold  
80:     - name: Install CPU-only torch (no CUDA wheels on a CPU runner)  
81:     # torch/torchvision are NOT in pyproject deps ? their CPU/CUDA wheels  
84:     run: python -m pip install torch torchvision --index-url  
https://download.pytorch.org/whl/cpu  
101: # (only numpy, no cv2/torch), so this is a FAST, focused job ? it does NOT run
```

45. assistant (2026-06-21T13:54:37.545Z)

Note: the engine's `requirements.txt` actually pins `torch>=2.12.0` (newer than the "2.6" in the task framing, but the dual-env conflict is identical ? main env is modern torch 2.x, WASB needs torch 1.11). The base.py already documents the ONNX-runner de-risk path. Let me read that documented seam to ground the ONNX-export option precisely.

46. user (2026-06-21T13:54:39.335Z)

```
1:"""Abstract base for detector runners (TrackNet, WASB, future native, etc.).  
2-  
3-This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1  
--  
7-The ABC lets us:  
8-- Keep the current WSL adapters working unchanged for Windows users.  
9-- Swap in (or A/B test) a Linux-native implementation later with zero changes to
```

```

10- the hybrid segmenters or the model-agnostic windowing brain.
11-- Provide a single place to evolve the healthcheck / error contract and thread
--
34- deliberately static and model-agnostic). If the new detector needs different
35- windowing, it can implement its own or we generalize later.
36:4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37: registry never pulls torch/tf on a machine that doesn't have them).
38-5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39- DetectorRunner instances (they already do the right thing via duck-typing + the
--
44- windowing helpers directly).
45-
46:Optional Linux-native path (the main de-risk goal):
47:A LinuxNativeRunner (or ONNXRunner) would:
48-- Skip all wsl / _stage / conda activate.
49:- Load weights locally (torch or onnx) or call a native binary.
50-- Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51:- healthcheck can just check "weights file exists and torch importable + CUDA?".
52-This removes the WSL requirement for Linux users / future SaaS workers while
53:keeping the exact same candidate quality characteristics (or better, once native
54:weights are available).
55-
--
96-     """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.
97-
98:   Lives beside (not on) DetectorRunner: a future Linux-native runner must not
99:   inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
100-   and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).

```

47. assistant (2026-06-21T13:54:42.446Z)

I have everything needed. The architecture already anticipates an ONNXRunner that loads weights with no torch dependency and emits the identical CSV. This is the cleanest path for a downloadable app. Let me synthesize the research into the structured output.

48. user (2026-06-21T13:56:36.521Z)

Structured output provided successfully

49. assistant (2026-06-21T13:56:46.382Z)

Research complete. I've returned the RESEARCH schema via StructuredOutput. Key findings grounded in code and web sources:

- ****The dual-env clash is real and unavoidable in one interpreter:**** engine main env pins `torch>=2.12.0` (`requirements.txt`), WASB-SBDT's Dockerfile pins `torch==1.11.0+cu113` + `timm==0.6.5`.
- ****But the codebase already isolates it behind a subprocess seam**** (`backend/pipeline/detectors/native\_wasb\_runner.py` shells `WASB\_PYTHON`), and `base.py:46-54` explicitly pre-designed an `ONNXRunner` that emits the identical CSV.
- ****The Anaconda licensing trap is the `defaults` channel, not conda itself**** ? conda-forge/miniforge/micromamba/conda-pack are all MIT/BSD and free for commercial redistribution.
- ****WASB is a vanilla CNN (HRNet via timm + a Python tracker), MIT-licensed**** ? so both the torch-2.x port and ONNX export are low-op-risk.

My recommendation is layered: (f) ship a ****torch-free LIGHT tier**** (Gemini, already a hard dep) as the reliable default download for the non-tech persona, and for the opt-in HEAVY on-device tier, ****('d') export WASB to ONNX + onnxruntime**** ? the path the ABC was built for, which dissolves the dual-env problem entirely (no torch 1.11 OR 2.x on the WASB path, ~tens-of-MB runtime, CPU/CUDA/DirectML/CoreML from one artifact). Bridge via (b) micromamba-from-conda-forge reusing the existing subprocess seam while ONNX is validated against the owner's byte-parity gate. Avoid Docker (e) as the local default.